

Contents

1	Sequence Diagram - Admin Dashboard	6
1.1	Overview	6
1.2	Participants	6
1.3	Flow	8
1.3.1	Part 1 - Dashboard Summary Load	8
1.3.2	Part 2 - Audit Log Pagination	8
1.3.3	Part 3 - Stats Aggregation Trigger	8
1.4	Error Paths	9
1.5	Key Implementation Details	9
2	Sequence Diagram - Admin Model Moderation	9
2.1	Overview	9
2.2	Participants	9
2.3	Flow	9
2.3.1	1. Load Pending Queue	9
2.3.2	2. Approve a Model	11
2.3.3	3. Reject a Model	11
2.4	Error Paths	11
2.5	Key Implementation Details	11
3	Sequence Diagram - Admin User Moderation	11
3.1	Overview	11
3.2	Participants	13
3.3	Flow	13
3.3.1	1. Load Pending Applications	13
3.3.2	2. Approve an Application	13
3.3.3	3. Reject an Application	13
3.4	Error Paths	14
3.5	Key Implementation Details	14
4	Sequence Diagram - Cart Management	14
4.1	Overview	14
4.2	Participants	14
4.3	Flow	14
4.3.1	Part 1 - Add to Cart	14
4.3.2	Part 2 - View Cart	16
4.3.3	Part 3 - Remove Item	16
4.3.4	Part 4 - Clear Cart	16
4.4	Error Paths	16
4.5	Key Implementation Details	16
5	Sequence Diagram - Catalog Search	16
5.1	Overview	18
5.2	Participants	18
5.3	Flow	18
5.3.1	1. Search Input	18
5.3.2	2. User Context Check	18
5.3.3	3. WHERE Clause Construction	18
5.3.4	4. Sort Order	18
5.3.5	5. Pagination	18
5.3.6	6. Database Query	18
5.3.7	7. S3 URL Signing	19
5.3.8	8. Response	19

5.4	Key Implementation Details	19
6	Sequence Diagram - Checkout	19
6.1	Overview	19
6.2	Participants	19
6.3	Flow	19
6.3.1	1. Checkout Trigger	19
6.3.2	2. Model Validation	19
6.3.3	3. Duplicate Purchase Check	21
6.3.4	4. Order Creation	21
6.3.5	5. Midtrans Snap Token	21
6.3.6	6. Payment Popup	21
6.4	Error Paths	21
6.5	Key Implementation Details	21
7	Sequence Diagram - Collections	22
7.1	Overview	22
7.2	Participants	22
7.3	Flow	22
7.3.1	Part 1 - List Collections	22
7.3.2	Part 2 - Create Collection	22
7.3.3	Part 3 - View Collection Detail	22
7.3.4	Part 4 - Add a Model	24
7.3.5	Part 5 - Remove a Model	24
7.4	Error Paths	24
7.5	Key Implementation Details	24
8	Sequence Diagram - Dexie Tagline and Picks	24
8.1	Overview	24
8.2	Participants	24
8.3	Flow	25
8.3.1	Part 1 - Tagline Generation (Catalog Route)	25
8.3.2	Part 2 - Personalized Picks (Home Page)	25
8.4	Key Implementation Details	27
9	Sequence Diagram - Dexie Toggle	27
9.1	Overview	27
9.2	Participants	27
9.3	Flow	27
9.3.1	Part 1 - Disable Dexie	27
9.3.2	Part 2 - Re-enable Dexie	27
9.4	Key Implementation Details	29
10	Sequence Diagram - Follow and Unfollow	29
10.1	Overview	29
10.2	Participants	29
10.3	Flow	29
10.3.1	Part 1 - Load Profile and Follow Stats	29
10.3.2	Part 2 - Follow	31
10.3.3	Part 3 - Unfollow	31
10.4	Error Paths	31
10.5	Key Implementation Details	31
11	Sequence Diagram - Google OAuth Login	31
11.1	Overview	31

11.2	Participants	33
11.3	Flow	33
11.3.1	1. OAuth Prompt	33
11.3.2	2. Token Delivery	33
11.3.3	3. Token Verification	33
11.3.4	4. User Lookup - Three Cases	33
11.3.5	5. JWT and Cookie	33
11.3.6	6. Conditional Redirect	33
11.4	Error Paths	33
11.5	Key Implementation Details	34
12	Sequence Diagram - Local Login	34
12.1	Overview	34
12.2	Participants	34
12.3	Flow	34
12.3.1	1. Form Submission	34
12.3.2	2. Field Validation	34
12.3.3	3. Credential Lookup	36
12.3.4	4. Password Verification	36
12.3.5	5. Last Login Update	36
12.3.6	6. JWT Signing and Cookie	36
12.3.7	7. S3 URL Signing	36
12.3.8	8. Response and Redirect	36
12.4	Error Paths	36
12.5	Key Implementation Details	36
13	Sequence Diagram - Model Delete	38
13.1	Overview	38
13.2	Participants	38
13.3	Flow	38
13.3.1	1. Delete Request	38
13.3.2	2. Model Lookup	38
13.3.3	3. Admin Path	38
13.3.4	4. Artist (Owner) Path	38
13.3.5	5. Deletion with Cascade	38
13.3.6	6. Response	39
13.4	Error Paths	39
13.5	Key Implementation Details	39
14	Sequence Diagram - Model Detail	39
14.1	Overview	39
14.2	Participants	39
14.3	Flow	41
14.3.1	1. Model Lookup	41
14.3.2	2. Purchase Check (Authenticated Users Only)	41
14.3.3	3. URL Signing	41
14.3.4	4. Response	41
14.4	Key Implementation Details	41
15	Sequence Diagram - Model Download	41
15.1	Overview	41
15.2	Participants	43
15.3	Flow	43
15.3.1	1. Download Request	43
15.3.2	2. Model Lookup	43

15.3.3	3. Purchase Verification	43
15.3.4	4. Presigned Download URL	43
15.3.5	5. Download Initiation	43
15.4	Error Paths	43
15.5	Key Implementation Details	44
16	Sequence Diagram - Model Review	44
16.1	Overview	44
16.2	Participants	44
16.3	Flow	44
16.3.1	1. Review Form	44
16.3.2	2. Submission	44
16.3.3	3. Rating Validation	44
16.3.4	4. Purchase Verification	44
16.3.5	5. Review Creation	46
16.3.6	6. Rating Recalculation	46
16.3.7	7. Response	46
16.4	Error Paths	46
16.5	Key Implementation Details	46
17	Sequence Diagram - Model Update	46
17.1	Overview	46
17.2	Participants	46
17.3	Flow	47
17.3.1	1. Edit Form Load	47
17.3.2	2. Changes and Submission	47
17.3.3	3. Authorization Check	47
17.3.4	4. Update Execution	47
17.3.5	5. Response	47
17.4	Error Paths	47
17.5	Key Implementation Details	47
18	Sequence Diagram - Model Upload	49
18.1	Overview	49
18.2	Participants	49
18.3	Flow	49
18.3.1	1. Presigned URL for GLB File	49
18.3.2	2. Direct Upload to S3	49
18.3.3	3. Presigned URL for Preview Image	49
18.3.4	4. Preview Upload	51
18.3.5	5. Model Metadata Submission	51
18.3.6	6. Embedding Generation (Fire and Forget)	51
18.3.7	7. Response	51
18.4	Error Paths	51
18.5	Key Implementation Details	51
19	Sequence Diagram - Notifications	51
19.1	Overview	53
19.2	Participants	53
19.3	Flow	53
19.3.1	Part 1 - Load Notifications	53
19.3.2	Part 2 - Mark Single as Read	53
19.3.3	Part 3 - Mark All as Read	53
19.4	Key Implementation Details	53

20 Sequence Diagram - Payment Webhook	53
20.1 Overview	55
20.2 Participants	55
20.3 Flow	55
20.3.1 1. Webhook Delivery	55
20.3.2 2. Order Lookup	55
20.3.3 3. Payment Record Upsert	55
20.3.4 4. Status Branching	55
20.3.5 5. Response	56
20.4 Key Implementation Details	56
21 Sequence Diagram - Print Job Management	56
21.1 Overview	56
21.2 Participants	56
21.3 Flow	56
21.3.1 Part 1 - View Incoming Jobs	56
21.3.2 Part 2 - Accept a Job	58
21.3.3 Part 3 - Ship the Job	58
21.3.4 Part 4 - Mark Delivered	58
21.4 Key Implementation Details	58
22 Sequence Diagram - Print Order	58
22.1 Overview	58
22.2 Participants	60
22.3 Flow	60
22.3.1 Part 1 - Browse Providers	60
22.3.2 Part 2 - Submit Print Order	60
22.4 Error Paths	61
22.5 Key Implementation Details	61
23 Sequence Diagram - User Registration	61
23.1 Overview	61
23.2 Participants	61
23.3 Flow	61
23.3.1 1. Form Submission	61
23.3.2 2. Field Validation	61
23.3.3 3. Password Hashing	63
23.3.4 4. User Creation	63
23.3.5 5. Response	63
23.4 Error Paths	63
23.5 Key Implementation Details	63
24 Sequence Diagram - Report Submission and Resolution	63
24.1 Overview	63
24.2 Participants	63
24.3 Flow	64
24.3.1 Phase 1 - User Submits a Report	64
24.3.2 Phase 2 - Admin Resolves the Report	64
24.4 Error Paths	64
24.5 Key Implementation Details	64
25 Sequence Diagram - Session Hydration	66
25.1 Overview	66
25.2 Participants	66
25.3 Flow	66

25.3.1	1. App Mount or Page Refresh	66
25.3.2	2. JWT Validation	66
25.3.3	3. Database Lookup	66
25.3.4	4. Security Sanitization	66
25.3.5	5. Token Refresh	68
25.3.6	6. URL Signing	68
25.3.7	7. Response	68
25.4	Error Paths	68
25.5	Key Implementation Details	68
26	Sequence Diagram - Social Post	68
26.1	Overview	68
26.2	Participants	68
26.3	Flow	69
26.3.1	Part 1 - Load Feed	69
26.3.2	Part 2 - Create a Post	69
26.3.3	Part 3 - Like / Unlike	69
26.3.4	Part 4 - Add a Comment	69
26.4	Error Paths	71
26.5	Key Implementation Details	71
27	Sequence Diagram - User Review	71
27.1	Overview	71
27.2	Participants	71
27.3	Flow	71
27.3.1	Part 1 - Eligibility Check	71
27.3.2	Part 2 - Review Submission	73
27.4	Error Paths	73
27.5	Key Implementation Details	73
28	Sequence Diagram - Wishlist	73
28.1	Overview	75
28.2	Participants	75
28.3	Flow	75
28.3.1	Part 1 - Toggle Wishlist	75
28.3.2	Part 2 - Wishlist Page	75
28.4	Key Implementation Details	75

1 Sequence Diagram - Admin Dashboard

UML Type: Sequence Diagram **Category:** Admin **Source:** docs/diagrams/sequence/admin/sequence_admin_dash

1.1 Overview

This diagram covers three distinct admin actions available from the admin dashboard: loading the summary overview, paginating through the audit log, and manually triggering a platform stats aggregation.

1.2 Participants

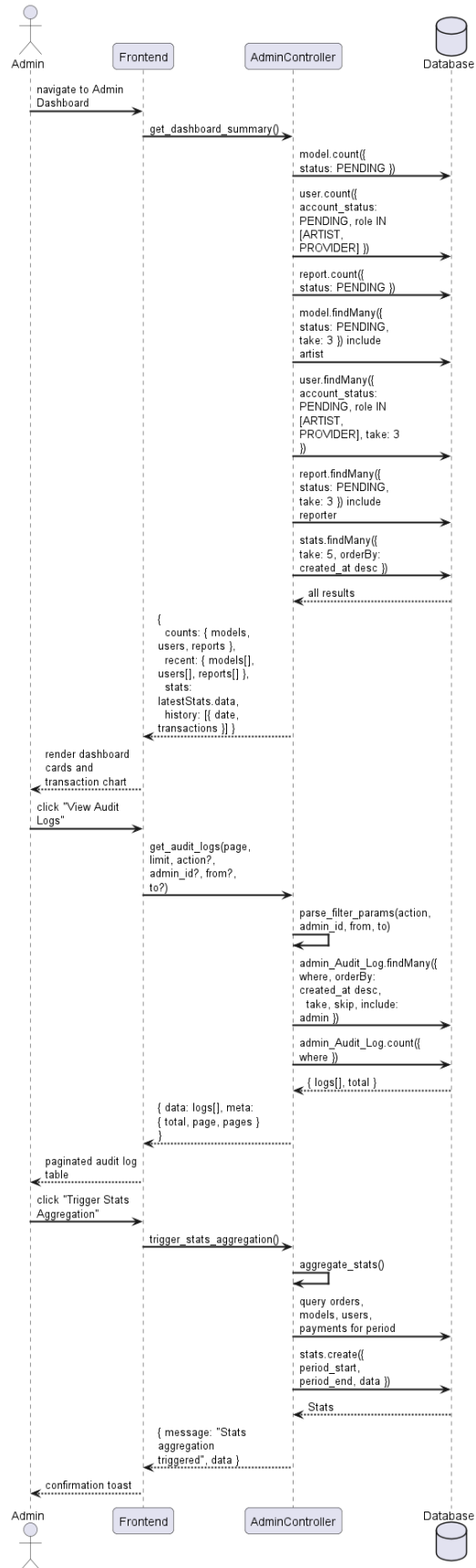


Figure 1: Sequence Diagram - Admin Dashboard

Participant	Role
Admin	The authenticated admin user
Frontend	The admin panel client
AdminController	Express route handler for admin endpoints
Database	PostgreSQL via Prisma

1.3 Flow

1.3.1 Part 1 - Dashboard Summary Load

Trigger: Admin navigates to the Admin Dashboard page.

The Frontend calls `get_dashboard_summary()`. AdminController fires several parallel database queries:

1. Count of models with `status = PENDING`
2. Count of users with `account_status = PENDING` and `role IN [ARTIST, PROVIDER]`
3. Count of reports with `status = PENDING`
4. Latest 3 pending models (including artist data)
5. Latest 3 pending user applications
6. Latest 3 pending reports (including reporter data)
7. Latest 5 stats records ordered by `created_at` descending

The results are compiled into a single response object:

```
{
  counts: { models, users, reports },
  recent: { models[], users[], reports[] },
  stats: latestStats.data,
  history: [{ date, transactions }]
}
```

The Frontend uses this to render the dashboard counter cards and the transaction history chart.

1.3.2 Part 2 - Audit Log Pagination

Trigger: Admin clicks “View Audit Logs”.

The Frontend calls `get_audit_logs(page, limit, action?, admin_id?, from?, to?)`.

AdminController parses the optional filter parameters and constructs a filtered query against the `Admin_Audit_Log` table, ordering by `created_at` descending. It also counts the total number of matching records for pagination metadata.

The response is:

```
{ data: logs[], meta: { total, page, pages } }
```

The Frontend renders a paginated table of log entries.

1.3.3 Part 3 - Stats Aggregation Trigger

Trigger: Admin clicks “Trigger Stats Aggregation”.

The Frontend calls `trigger_stats_aggregation()`. AdminController calls `aggregate_stats()`, which queries: - All orders, models, users, and payments for a defined period

It then creates a new **Stats** record in the database with { `period_start`, `period_end`, `data` }.

The response confirms the aggregation was triggered. The Frontend shows a confirmation toast.

1.4 Error Paths

This diagram does not include explicit error handling beyond the general authentication guard. All three actions require the user to have the **ADMIN** role, enforced by the role middleware before the controller runs.

1.5 Key Implementation Details

- The dashboard summary is a single controller action that executes multiple Prisma queries. In the current implementation these are executed sequentially, not in parallel. Each query is separate.
- The audit log uses cursor-based or offset-based pagination with `take` and `skip` parameters derived from `page` and `limit`.
- Stats aggregation can also be triggered by the cron scheduler automatically. The dashboard trigger is a manual on-demand option.

2 Sequence Diagram - Admin Model Moderation

UML Type: Sequence Diagram **Category:** Admin **Source:** docs/diagrams/sequence/admin/sequence_admin_mod

2.1 Overview

This diagram describes how an Admin reviews, approves, or rejects 3D models submitted by Artists. Models start in the **PENDING** state and must be approved before appearing in the public catalog.

2.2 Participants

Participant	Role
Admin	The authenticated admin user
Frontend	The admin panel client
AdminController	Express route handler for admin model actions
Database	PostgreSQL via Prisma

2.3 Flow

2.3.1 1. Load Pending Queue

The Admin navigates to the model moderation page. The Frontend calls `get_pending_models()`. Admin-Controller queries the database for all models with `status = PENDING`. The Frontend displays them in a review queue.

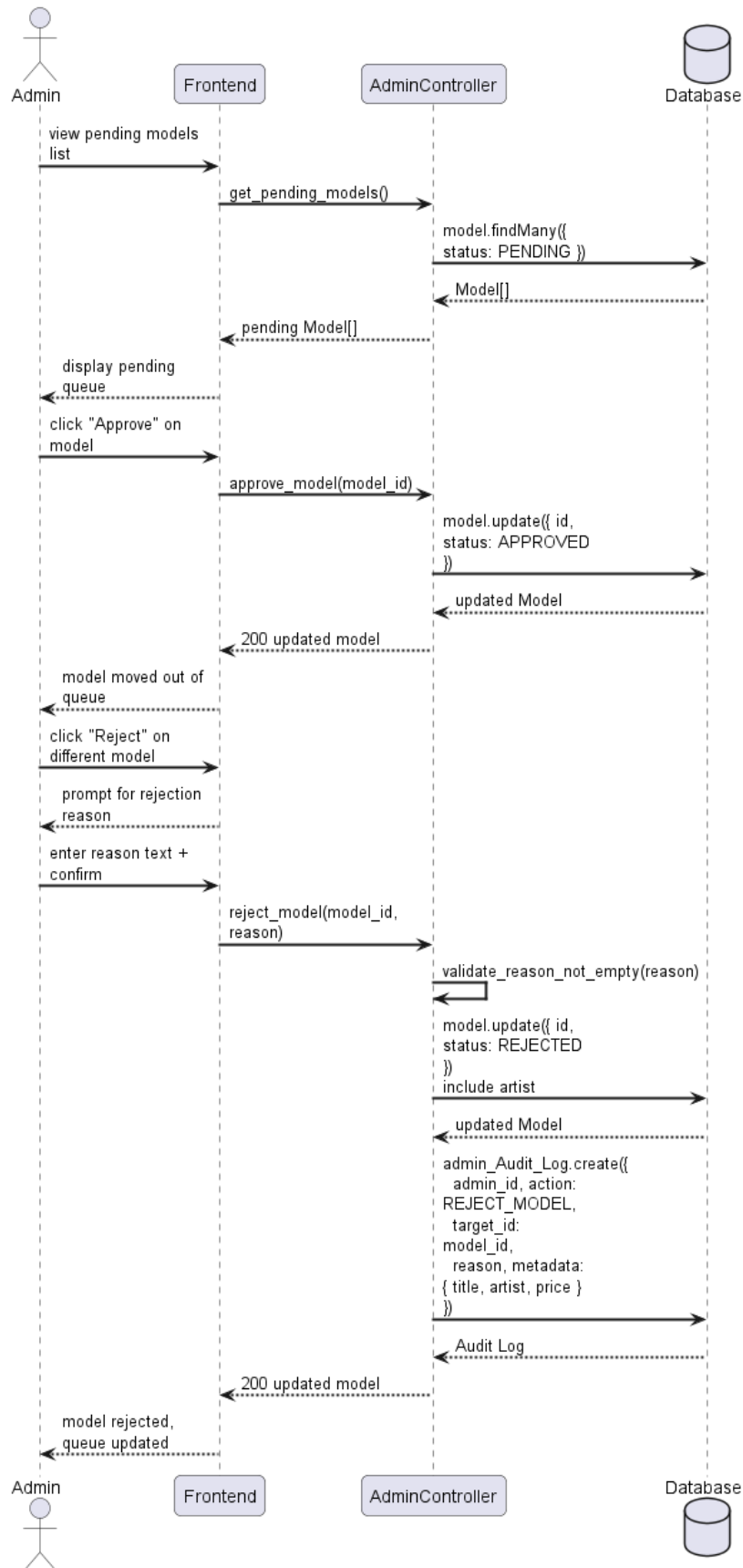


Figure 2: Sequence Diagram - Admin Model Moderation

2.3.2 2. Approve a Model

The Admin clicks “Approve” on a model in the queue.

The Frontend calls `approve_model(model_id)`. AdminController: 1. Calls `model.update({ id, status: APPROVED })`

Note: The approval action does not create an audit log entry in this diagram. Only rejections create audit log records, as approvals are less likely to be disputed.

The Frontend receives the updated model object and removes it from the pending queue.

2.3.3 3. Reject a Model

The Admin clicks “Reject” on a different model.

The Frontend prompts the Admin to enter a rejection reason.

The Admin enters text and confirms. The Frontend calls `reject_model(model_id, reason)`.

AdminController: 1. Validates that `reason` is not empty. 2. Calls `model.update({ id, status: REJECTED })`, including the artist in the fetched result. 3. Creates an `Admin_Audit_Log` record with: - `admin_id`: the acting admin’s user ID - `action`: `REJECT_MODEL` - `target_id`: the model’s ID - `reason`: the entered reason text - `metadata`: { `title`, `artist`, `price` }

The Frontend receives confirmation and removes the model from the queue.

2.4 Error Paths

Condition	HTTP Response
Rejection reason is empty	400 Bad Request (validated by controller)

2.5 Key Implementation Details

- Only rejections write audit log entries. This is because rejections are the actionable decision that needs accountability, while approvals are the expected default outcome.
- After rejection, the model remains in the database with `status = REJECTED`. The artist can see this state in their dashboard. The admin can choose to re-approve a rejected model at any time.
- The `metadata` field in the audit log preserves a snapshot of the model’s details at the time of action, ensuring the log is meaningful even if the model is later deleted.

3 Sequence Diagram - Admin User Moderation

UML Type: Sequence Diagram **Category:** Admin **Source:** docs/diagrams/sequence/admin/sequence_admin_user

3.1 Overview

This diagram describes how an Admin reviews, approves, or rejects pending Artist and Provider applications. Customers are auto-approved at registration; only role-upgrade applications from Artists and Providers require manual review.

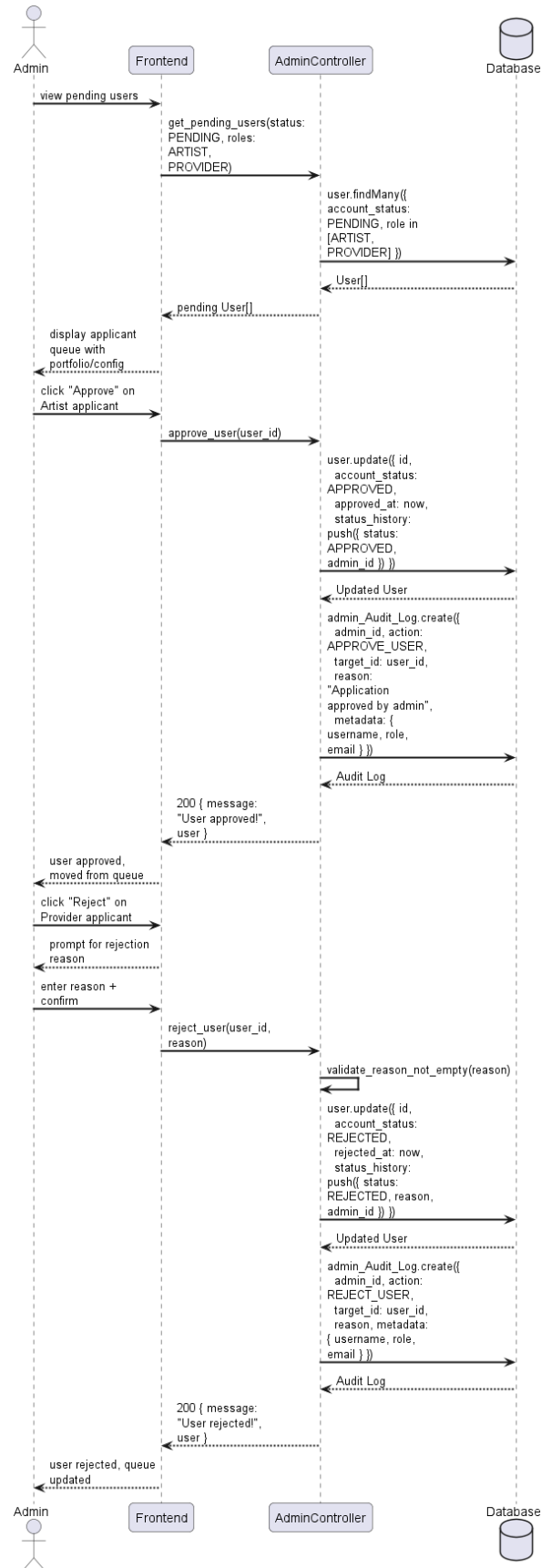


Figure 3: Sequence Diagram - Admin User Moderation

3.2 Participants

Participant	Role
Admin	The authenticated admin user
Frontend	The admin panel client
AdminController	Express route handler for admin user actions
Database	PostgreSQL via Prisma

3.3 Flow

3.3.1 1. Load Pending Applications

The Admin opens the user moderation view. The Frontend calls `get_pending_users(status: PENDING, roles: ARTIST, PROVIDER)`.

AdminController queries the database for users with `account_status = PENDING` and `role IN [ARTIST, PROVIDER]`. The Frontend displays the applicants alongside their portfolio or provider_config data.

3.3.2 2. Approve an Application

The Admin clicks “Approve” on an Artist applicant.

The Frontend calls `approve_user(user_id)`. AdminController:

1. Calls `user.update` setting:
 - `account_status = APPROVED`
 - `approved_at = now()`
 - `status_history: push({ status: APPROVED, admin_id })`
2. Creates an `Admin_Audit_Log` record with:
 - `action = APPROVE_USER`
 - `target_id = user_id`
 - `reason = "Application approved by admin"`
 - `metadata = { username, role, email }`

The Frontend receives confirmation and removes the user from the queue.

3.3.3 3. Reject an Application

The Admin clicks “Reject” on a Provider applicant.

The Frontend prompts the Admin for a rejection reason. After the Admin enters and confirms the reason, the Frontend calls `reject_user(user_id, reason)`.

AdminController:

1. Validates that `reason` is not empty.
2. Calls `user.update` setting:
 - `account_status = REJECTED`
 - `rejected_at = now()`
 - `status_history: push({ status: REJECTED, reason, admin_id })`
3. Creates an `Admin_Audit_Log` record with:
 - `action = REJECT_USER`
 - `target_id = user_id`
 - `reason = <entered reason>`
 - `metadata = { username, role, email }`

The Frontend receives confirmation and updates the queue.

3.4 Error Paths

Condition	HTTP Response
Rejection reason is empty	400 Bad Request

3.5 Key Implementation Details

- Both approval and rejection write audit log entries for user moderation. This differs from model moderation, where only rejections are logged.
- The `status_history` field is a `Json[]` array on the `User` table. Each approval or rejection appends a new entry, preserving the full history of status changes with timestamps and the acting admin's ID.
- After rejection, the applicant's `account_status` is set to `REJECTED`. They can see this in their profile settings. Whether they can re-apply is an application-level policy decision not enforced at the database level.
- Approved Artists and Providers gain access to role-specific features immediately after the database update. There is no additional activation step.

4 Sequence Diagram - Cart Management

UML Type: Sequence Diagram **Category:** Commerce **Source:** docs/diagrams/sequence/commerce/sequence_car

4.1 Overview

This diagram covers the full cart management lifecycle: adding an item, viewing the cart, removing an individual item, and clearing the entire cart.

4.2 Participants

Participant	Role
User	The authenticated user managing their cart
Frontend	The client
CartController	Express route handler for cart operations
StorageClient (S3)	MinIO for presigning preview images
Database	PostgreSQL via Prisma

4.3 Flow

4.3.1 Part 1 - Add to Cart

The user clicks “Add to Cart” on a model. The Frontend calls `add_to_cart(model_id, session_cookie)`.

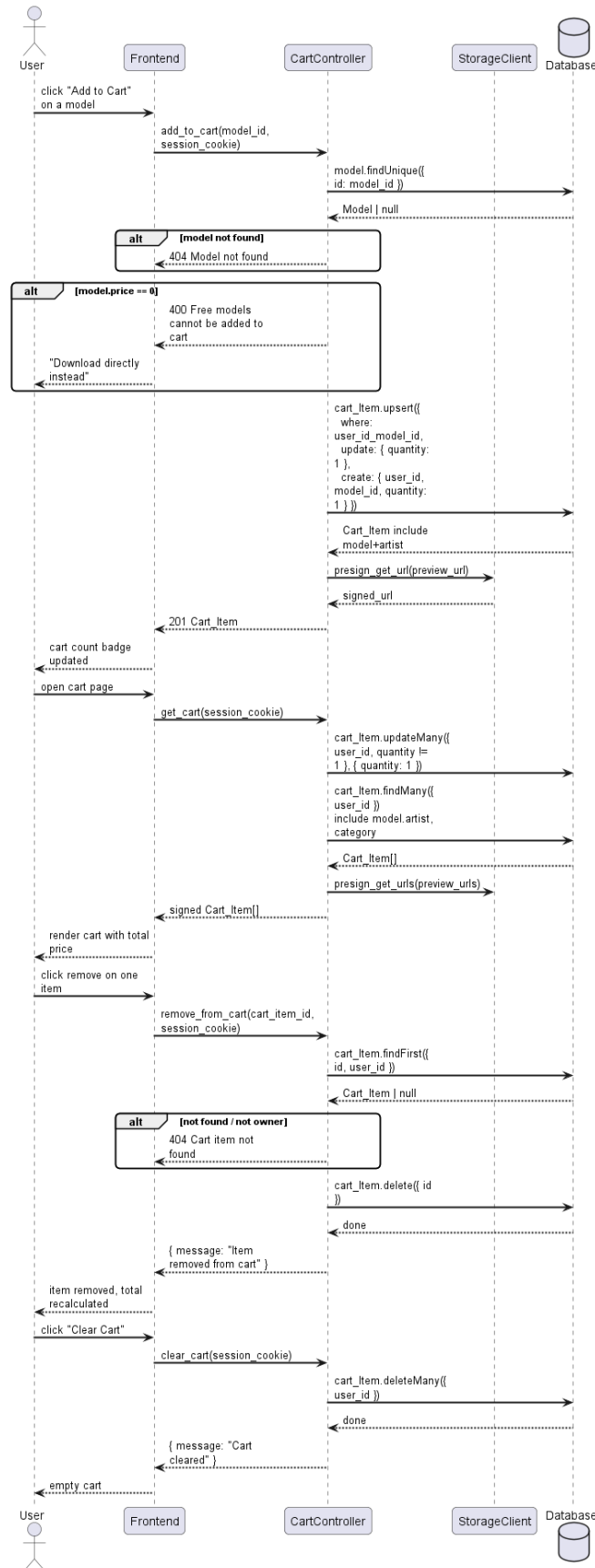


Figure 4: Sequence Diagram - Cart Management

CartController: 1. Looks up the model by ID. If not found: returns **404 Model not found**. 2. Checks `model.price`. If `price == 0`: returns **400 Free models cannot be added to cart** with a suggestion to download directly. 3. Calls `cart_Item.upsert` with `where: user_id_model_id`. If the item already exists, it stays at quantity 1. Otherwise a new record is created. 4. Presigns the model's `preview_url` via MinIO. 5. Returns **201 Cart_Item**. The Frontend updates the cart count badge.

4.3.2 Part 2 - View Cart

The user opens the cart page. The Frontend calls `get_cart(session_cookie)`.

CartController: 1. Normalizes all existing cart items by calling `cart_Item.updateMany({ user_id, quantity != 1 }, { quantity: 1 })` to enforce that all digital items have quantity 1. 2. Fetches all cart items for the user including model, artist, and category. 3. Presigns all preview URLs in batch. 4. Returns the signed cart items. The Frontend renders the cart with a total price.

4.3.3 Part 3 - Remove Item

The user clicks “Remove” on one cart item. The Frontend calls `remove_from_cart(cart_item_id, session_cookie)`.

CartController: 1. Looks up the item by `{ id, user_id }` to verify ownership. 2. If not found or not the owner: returns **404 Cart item not found**. 3. Deletes the item. 4. Returns confirmation. The Frontend updates the total.

4.3.4 Part 4 - Clear Cart

The user clicks “Clear Cart”. The Frontend calls `clear_cart(session_cookie)`.

CartController calls `cart_Item.deleteMany({ user_id })`. The Frontend renders an empty cart.

4.4 Error Paths

Condition	HTTP Response
Model not found	404 Not Found
Free model	400 Bad Request
Item not found or not owned	404 Not Found

4.5 Key Implementation Details

- The quantity enforcement step in `get_cart` ensures that even if a race condition or old data left a quantity > 1 , the system normalizes it. Digital goods are always quantity 1.
- The `upsert` pattern for adding items means clicking “Add to Cart” on an already-carted model is idempotent.
- Cart items are not deleted when a user completes checkout. The frontend must explicitly clear the cart after a successful payment confirmation.

5 Sequence Diagram - Catalog Search

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_catalog

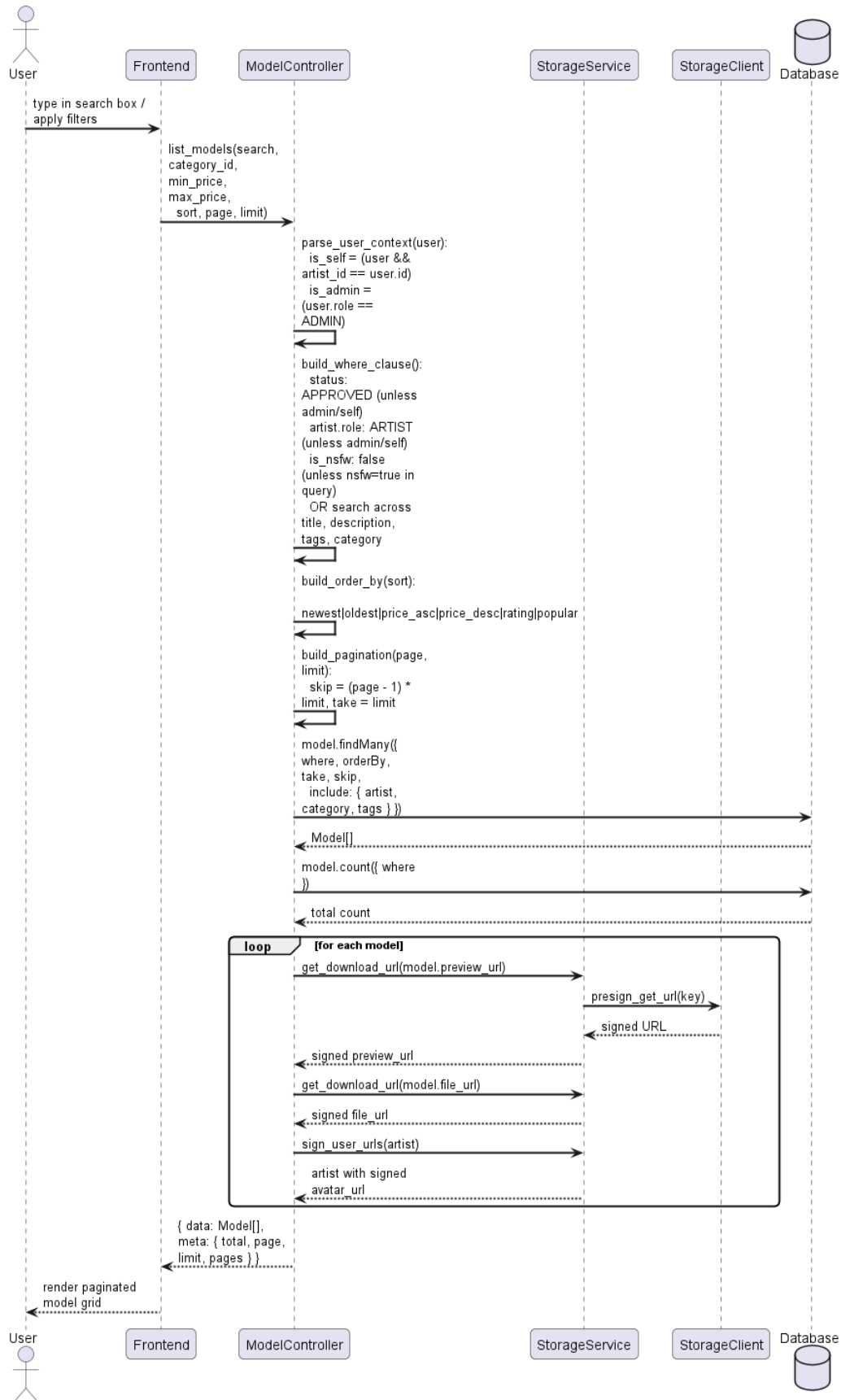


Figure 5: Sequence Diagram - Catalog Search

5.1 Overview

This diagram describes the full request-response cycle for the catalog model listing endpoint, including permission-aware filtering, sorting, pagination, and per-model S3 URL signing.

5.2 Participants

Participant	Role
User	Any visitor (authenticated or guest)
Frontend	The Next.js client rendering the catalog grid
ModelController	Express route handler for <code>/models</code>
StorageService	S3 URL presigner
StorageClient (S3)	MinIO S3 storage backend
Database	PostgreSQL via Prisma

5.3 Flow

5.3.1 1. Search Input

The user types in the search box or applies filters (category, price range, sort). The Frontend sends `list_models(search, category_id, min_price, max_price, sort, page, limit)`.

5.3.2 2. User Context Check

ModelController checks the caller's identity: - `is_self`: true if the user is authenticated and the `artist_id` filter matches their own ID - `is_admin`: true if the user has the `ADMIN` role

5.3.3 3. WHERE Clause Construction

Based on the context: - **Status filter**: Only `APPROVED` models are visible to regular users. Admins and self-views include all statuses. - **Artist role filter**: Only models from users with `role = ARTIST` are shown unless the caller is an admin or viewing their own models. - **NSFW filter**: NSFW models are hidden unless `show_nsfw=true` is in the query (respecting the user's preference). - **Search**: Performs an OR search across `title`, `description`, `tags.name`, and `category.name`. - **Price range**: Applies `gte` and `lte` filters on the `price` field. - **Category**: Filters by `category_id`.

5.3.4 4. Sort Order

The `sort` parameter maps to one of: `newest`, `oldest`, `price_asc`, `price_desc`, `rating`, or `popular`.

5.3.5 5. Pagination

`skip = (page - 1) * limit, take = limit`. The controller also runs `model.count({ where })` to return total count for pagination UI.

5.3.6 6. Database Query

`model.findMany` is called with the built `where`, `orderBy`, `take`, and `skip` along with `include: { artist, category, tags }`.

5.3.7 7. S3 URL Signing

For each returned model, ModelController calls StorageService to: - Presign the `preview_url` - Presign the `file_url` - Sign the artist's `avatar_url`

5.3.8 8. Response

The response is `{ data: Model[], meta: { total, page, limit, pages } }`. The Frontend renders the paginated model grid.

5.4 Key Implementation Details

- The loop for URL signing is per-model, which means N presign calls for N models. This is the current approach and is the primary source of catalog latency in production environments with many results.
- The `embedding` field is not used in keyword search. Keyword search is a direct Prisma filter on text fields. The `embedding` is used only in the Dexie AI semantic search flow.
- Free models (`price = 0`) appear in the catalog and can be downloaded without checkout.

6 Sequence Diagram - Checkout

UML Type: Sequence Diagram **Category:** Commerce **Source:** docs/diagrams/sequence/commerce/sequence_checkout

6.1 Overview

This diagram describes the digital asset checkout flow. It covers order creation, duplicate purchase prevention, Midtrans Snap API integration, and the final payment popup presentation.

6.2 Participants

Participant	Role
Customer	The authenticated user checking out
Frontend	The client
OrderController	Express route handler for checkout
OrderService	Business logic for order creation
MidtransClient	Midtrans Snap payment gateway API client
Database	PostgreSQL via Prisma

6.3 Flow

6.3.1 1. Checkout Trigger

The Customer clicks “Checkout” from the cart. The Frontend reads the cart items (list of `model_ids`) and sends `checkout({ items: [{ model_id, quantity }], session_cookie)` to OrderController.

6.3.2 2. Model Validation

OrderService queries `model.findMany({ id IN model_ids })` to validate all models exist and are APPROVED.

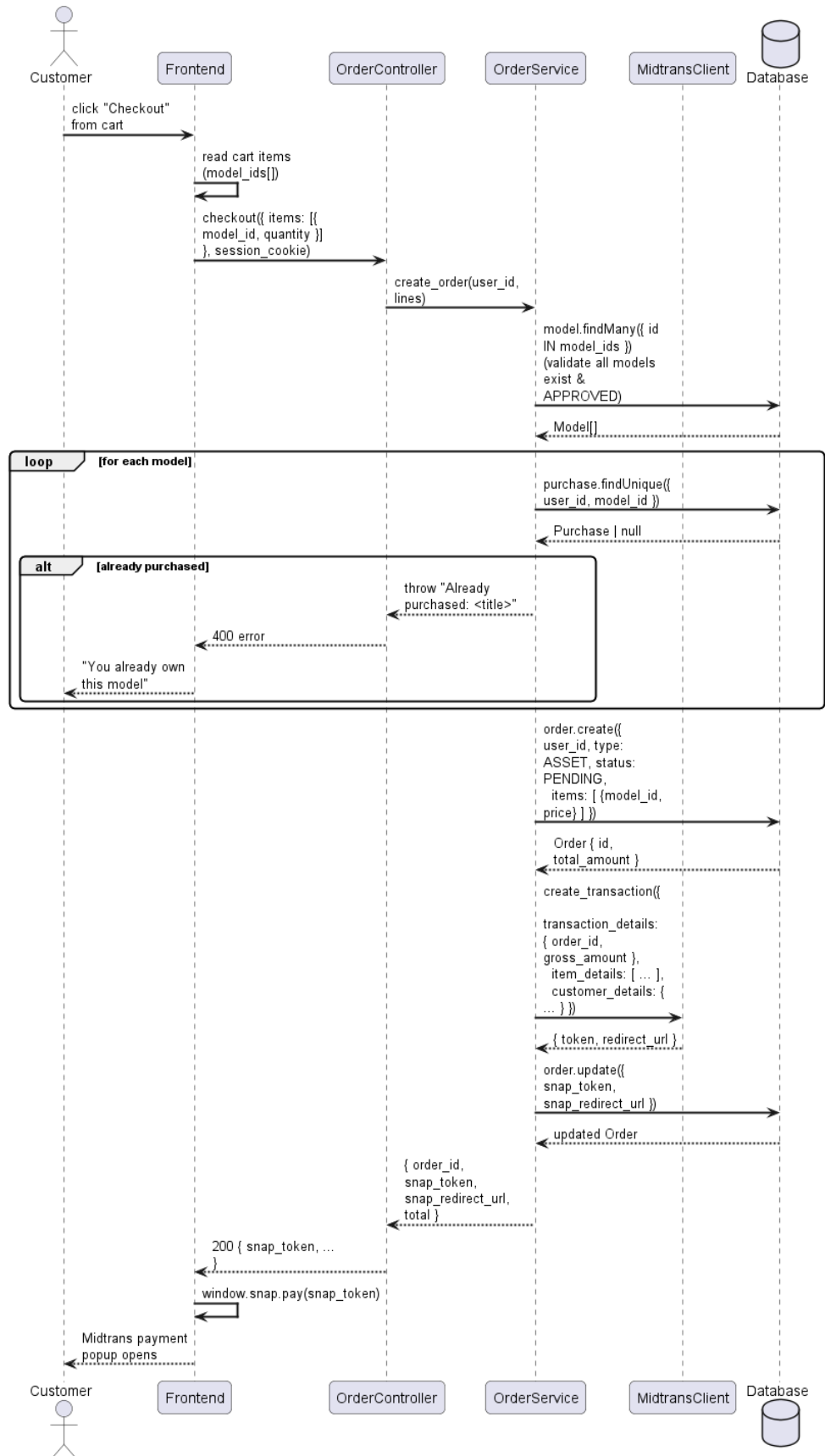


Figure 6: Sequence Diagram - Checkout

6.3.3 3. Duplicate Purchase Check

For each model in the order, OrderService checks `purchase.findUnique({ user_id, model_id })`.

If any model has already been purchased by this user: - OrderService throws an error: “Already purchased: [model title]”. - OrderController returns **400**. The Frontend shows the error. Flow terminates.

6.3.4 4. Order Creation

OrderService creates the order:

```
order.create({
  user_id,
  type: ASSET,
  status: PENDING,
  items: [ { model_id, price } for each model ]
})
```

The database returns the order with its computed `total_amount`.

6.3.5 5. Midtrans Snap Token

OrderService calls the Midtrans client to create a transaction:

```
{
  transaction_details: { order_id, gross_amount },
  item_details: [...],
  customer_details: {...}
}
```

Midtrans returns `{ token, redirect_url }`.

OrderService updates the order with `{ snap_token, snap_redirect_url }`.

6.3.6 6. Payment Popup

OrderController returns **200** `{ snap_token, snap_redirect_url, total }`. The Frontend calls `window.snap.pay(snap_token)`, which opens the Midtrans payment popup in the browser.

6.4 Error Paths

Condition	HTTP Response
Empty items array	400 Bad Request
Model already purchased	400 Already purchased
Midtrans gateway error	500 (implied rollback)

6.5 Key Implementation Details

- The `type: ASSET` distinguishes this from a `PRINT_JOB` order.
- The Midtrans Snap token is stored on the `Order` so the frontend can re-open the payment popup if the user closes it before completing payment.
- After the popup completes (success or failure), Midtrans sends a webhook to `/orders/notification`. The actual Purchase creation happens in the webhook handler, not here.

- The cart is NOT automatically cleared here. The frontend must handle clearing the cart after receiving confirmation of payment success from Midtrans.

7 Sequence Diagram - Collections

UML Type: Sequence Diagram **Category:** Commerce **Source:** docs/diagrams/sequence/commerce/sequence_col

7.1 Overview

This diagram covers the full collections feature: listing all collections, creating a new collection, viewing a collection's contents, adding a model to a collection, and removing a model from a collection.

7.2 Participants

Participant	Role
User	The authenticated user
Frontend	The client
CollectionController	Express route handler for collection operations
Database	PostgreSQL via Prisma

7.3 Flow

7.3.1 Part 1 - List Collections

The user opens “My Collections”. The Frontend calls `get_collections(session_cookie)`.

CollectionController queries `collection.findMany({ user_id })` including item counts. Returns the collection grid.

7.3.2 Part 2 - Create Collection

The user clicks “New Collection”. The Frontend shows a form for name, description, and visibility.

The user submits `{ name, description, is_public: true }`. The Frontend calls `create_collection({ name, description, is_public })`.

CollectionController calls `collection.create({ user_id, name, description, is_public })`. Returns **201 Collection**. The new collection card appears in the UI.

7.3.3 Part 3 - View Collection Detail

The user opens a collection. The Frontend calls `get_collection(collection_id)`.

CollectionController queries `collection.findUnique({ id })` including items and their associated models (with artist and tags).

Privacy Gate: If the collection is private (`is_public = false`) AND the requesting user is not the owner: returns **403 Forbidden**. Flow terminates.

Otherwise, returns the collection with items. The Frontend renders the model grid.

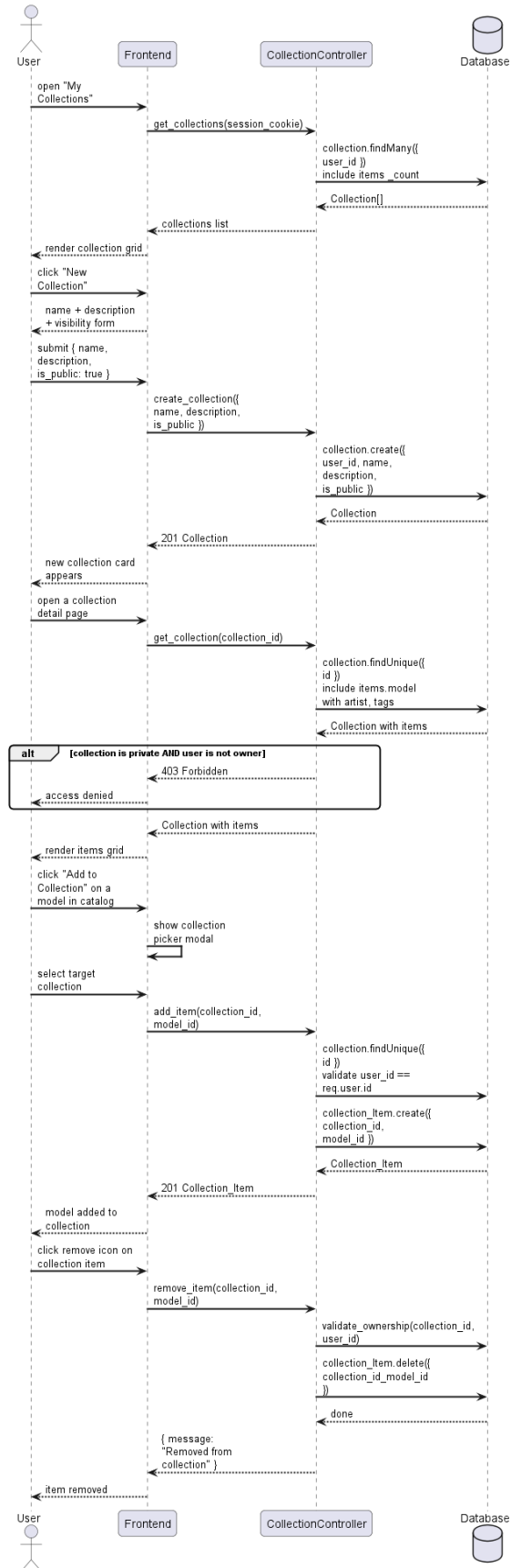


Figure 7: Sequence Diagram - Collections

7.3.4 Part 4 - Add a Model

The user clicks “Add to Collection” on a catalog model. The Frontend shows a collection picker modal.

The user selects a target collection. The Frontend calls `add_item(collection_id, model_id)`.

CollectionController: 1. Validates that the collection belongs to the requesting user. 2. Calls `collection_item.create({ collection_id, model_id })`. 3. Returns **201 Collection_Item**. The model is added to the collection.

7.3.5 Part 5 - Remove a Model

The user clicks the remove icon on an item. The Frontend calls `remove_item(collection_id, model_id)`.

CollectionController: 1. Validates ownership of the collection. 2. Calls `collection_item.delete({ collection_id_model_id })`. 3. Returns confirmation. The Frontend removes the item.

7.4 Error Paths

Condition	HTTP Response
Private collection viewed by non-owner	403 Forbidden
Ownership validation fails	403 or 404 (implied)

7.5 Key Implementation Details

- The `is_public` flag allows users to share their collections with others via a direct link. Only the owner can see private collections.
- `Collection_Item` has a unique constraint on `(collection_id, model_id)`, preventing the same model from being added twice.
- Both the `Collection` and `Collection_Item` are cascade-deleted when the parent collection is deleted, or the `Collection_Item` is cascade-deleted when the referenced model is deleted.

8 Sequence Diagram - Dexie Tagline and Picks

UML Type: Sequence Diagram **Category:** Dexie **Source:** docs/diagrams/sequence/dexie/sequence_dexie_tagline

8.1 Overview

This is the most complex diagram in the Dexie AI feature. It covers two combined flows: generating a contextual AI tagline as the user navigates to a model detail page, and serving personalized model picks when the user returns to the home page.

8.2 Participants

Participant	Role
User	The authenticated user
Frontend (DexieContext)	The React context managing Dexie state

Participant	Role
DexieController	Express route handler for Dexie endpoints
DexieService	Business logic for tagline and picks generation
EmbeddingService	Semantic embedding generator
GeminiClient	Google Gemini Flash API client
Database	PostgreSQL via Prisma

8.3 Flow

8.3.1 Part 1 - Tagline Generation (Catalog Route)

Trigger: User navigates to `/catalog/:model_id`.

1. The Frontend's `DexieContext` evaluates the path and constructs a context key: `{ ctx: "catalog", tag: model_id }`.
2. The Frontend calls `get_tagline(ctx: catalog, tag: model_id, session?)`.

`DexieController` checks if Dexie is enabled for the user via `user.dexie_enabled`. - If disabled: returns `{ enabled: false, message: null }`. Dexie stays hidden.

`DexieService` checks the **per-user cache** (1-hour TTL) for the user's cached tagline. - **Cache hit:** Returns the cached message immediately. - **Cache miss:** Calls the Gemini Flash API with a prompt at temperature 0.9.

If Gemini returns a 503 (rate limit): - `DexieService` performs an exponential backoff retry (max 2 retries). - If still failing: returns a random `FALLBACK_MESSAGE`.

On success: - Sets the **per-user cache** (1-hour TTL). - Sets the **global cache** for the same context (6-hour TTL).

`DexieController` returns `{ enabled: true, message }`.

The Frontend deduplicates the message against a session-scoped `seenMessages` Set: - If seen: stays quiet (shows nothing). - If new: displays the Dexie message bubble.

8.3.2 Part 2 - Personalized Picks (Home Page)

Trigger: User navigates to `/` after seeing a new Dexie message.

The Frontend calls `get_picks(limit: 8)`. `DexieController` calls `get_dexie_picks(user_id, { limit: 8 })`.

`DexieService` fetches user taste signals: - Recent 5 wishlist items (with model tags and category) - Recent 5 purchase items (with model tags and category)

If no taste data (new user): - Fetches the newest 8 APPROVED models as fallback picks.

If taste data exists: 1. Combines the tags and categories of wishlisted and purchased models into a single text string. 2. Calls `EmbeddingService` to embed the text using `all-MiniLM-L6-v2`, producing a 384-dimension query vector. 3. `EmbeddingService` runs a raw SQL cosine similarity query against the `embedding` column in the `Model` table. 4. The database returns models sorted by cosine similarity to the user's taste vector.

`DexieController` returns `{ enabled: true, picks }`. The Frontend renders the "Dexie's Picks" carousel section.

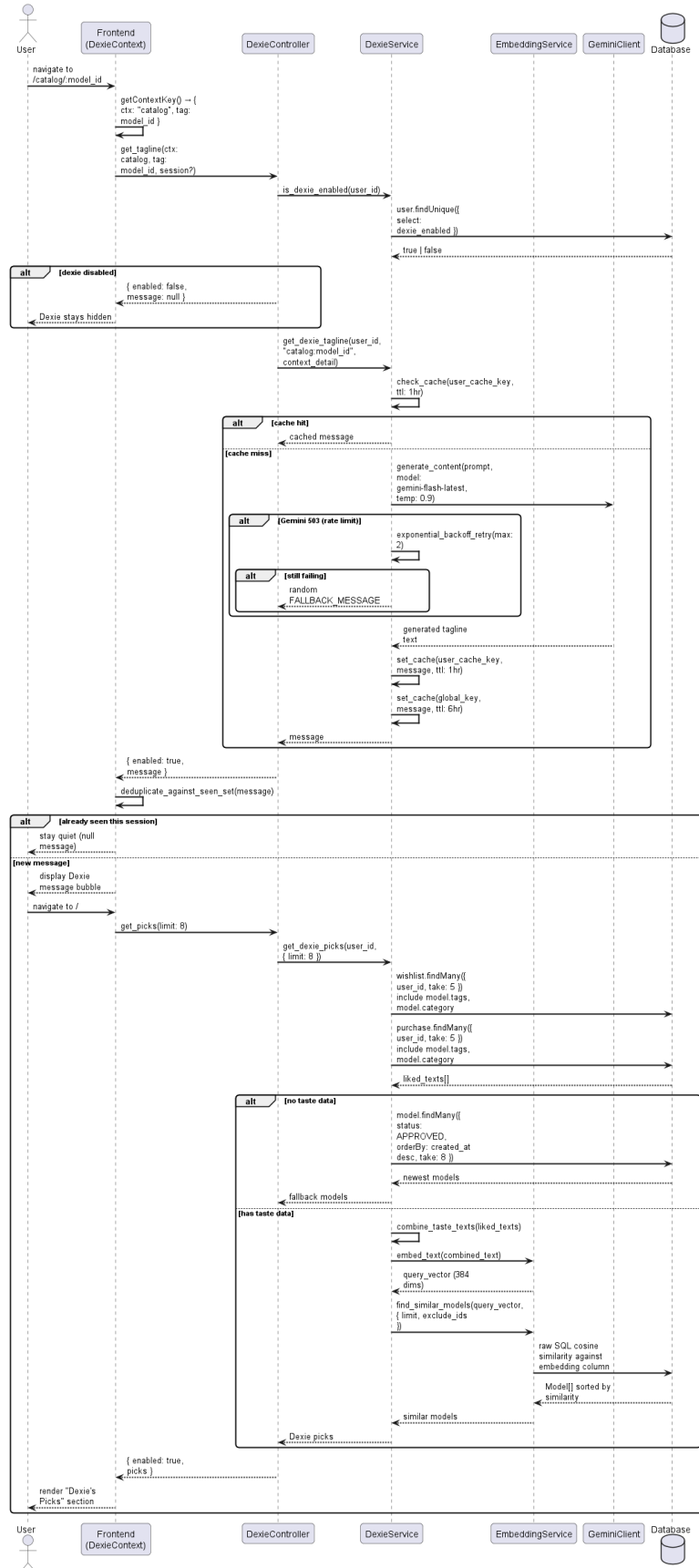


Figure 8: Sequence Diagram - Dexie Tagline and Picks

8.4 Key Implementation Details

- The dual-cache strategy (per-user 1hr, global 6hr) balances personalization with API cost. Frequent users get fresh content; infrequent users see globally cached responses.
- The `seenMessages` Set in the Frontend prevents the same AI message from being shown more than once per session, avoiding repetitive UI.
- The cosine similarity query is a raw SQL call because Prisma does not natively support `pgvector` operations. The `EmbeddingService` executes a parameterized `SELECT` with `<->` or `<=>` operators.
- Wishlist and purchase data are used as implicit “taste” signals, avoiding the need for explicit preference collection.

9 Sequence Diagram - Dexie Toggle

UML Type: Sequence Diagram **Category:** Dexie **Source:** docs/diagrams/sequence/dexie/sequence_dexie_togg

9.1 Overview

This diagram shows how a user enables or disables the Dexie AI assistant from their settings page. The toggle persists to the database and immediately updates the frontend’s Dexie state.

9.2 Participants

Participant	Role
User	The authenticated user
Frontend	The client
DexieController	Express route handler for the toggle endpoint
Database	PostgreSQL via Prisma

9.3 Flow

9.3.1 Part 1 - Disable Dexie

The user opens the Settings page. The Frontend renders a toggle switch showing Dexie’s current on/off state.

User flips toggle to OFF:

1. Frontend calls `toggle_dexie(enabled: false, session_cookie)`.
2. DexieController validates that `enabled` is a boolean.
3. DexieController calls `user.update({ id, dexie_enabled: false })`.
4. Returns `200 { message: “Dexie will quietly wait...”, dexie_enabled: false }`.
5. Frontend calls `DexieContext.setEnabled(false)` and clears the current message (`setMessage(null)`).
6. The Dexie bubble disappears and the toggle shows OFF state.

9.3.2 Part 2 - Re-enable Dexie

User flips toggle to ON:

1. Frontend calls `toggle_dexie(enabled: true, session_cookie)`.
2. DexieController calls `user.update({ id, dexie_enabled: true })`.

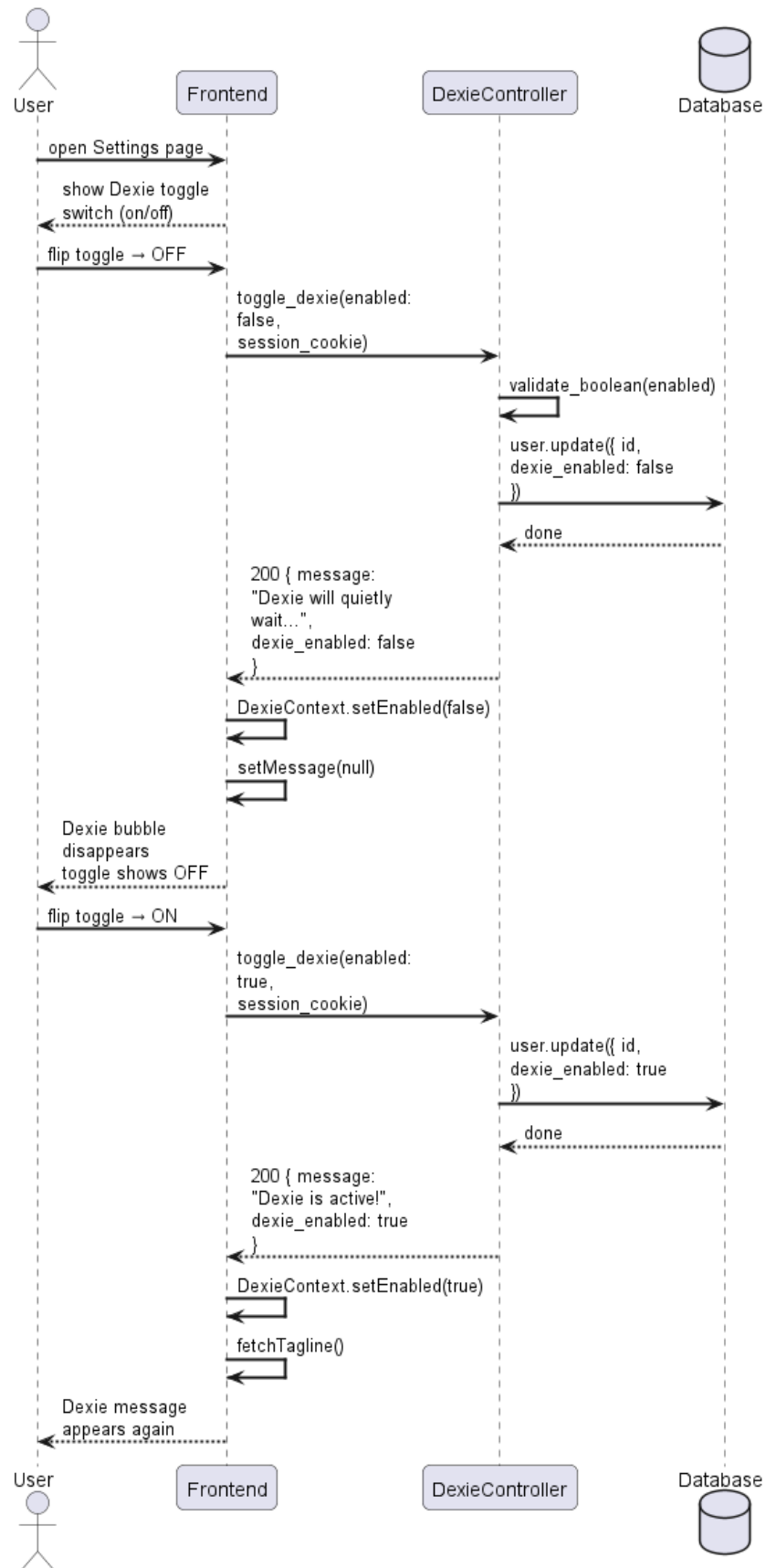


Figure 9: Sequence Diagram - Dexie Toggle

3. Returns **200 { message: “Dexie is active!”, dexie_enabled: true }**.
 4. Frontend calls `DexieContext.setEnabled(true)` and immediately calls `fetchTagline()` to surface a new Dexie message.
 5. The Dexie bubble appears again.
-

9.4 Key Implementation Details

- The `dexie_enabled` field is a `Boolean` column on the `User` table in Prisma, defaulting to `true`.
- Disabling Dexie affects both the tagline and the picks feature. The `is_dexie_enabled` check in `DexieService` gates both flows.
- The toggle immediately triggers a tagline fetch on re-enable, ensuring the user sees a message promptly without needing to navigate away and back.
- This is a user preference, not a platform-wide setting. Each user controls their own Dexie experience independently.

10 Sequence Diagram - Follow and Unfollow

UML Type: Sequence Diagram **Category:** Social **Source:** docs/diagrams/sequence/social/sequence_follow_un

10.1 Overview

This diagram describes the follow and unfollow flows when a user visits another user’s profile, checks the follow state, and toggles it.

10.2 Participants

Participant	Role
User	The authenticated user performing the action
Frontend	The client
FollowController	Express route handler for follow operations
NotificationService	In-app notification creator
Database	PostgreSQL via Prisma

10.3 Flow

10.3.1 Part 1 - Load Profile and Follow Stats

The user visits an artist’s profile. The Frontend calls `get_follow_stats(target_user_id)`.

FollowController queries: - `follow.findMany({ following_id: target })` to get followers - `follow.findMany({ follower_id: target })` to get following

Returns follower/following counts and a boolean `is_following` flag. The Frontend renders the Follow button accordingly.

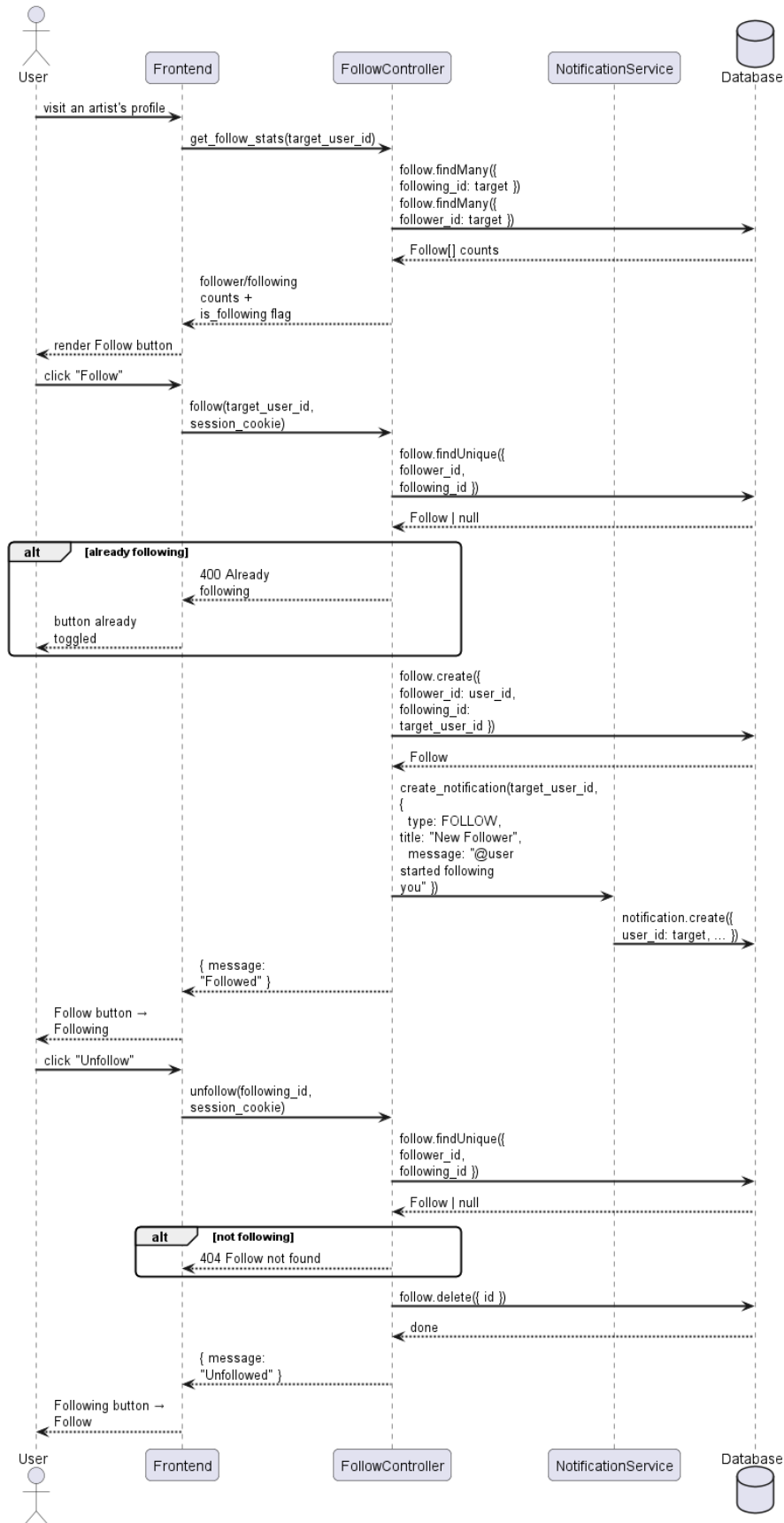


Figure 10: Sequence Diagram - Follow and Unfollow

10.3.2 Part 2 - Follow

The user clicks “Follow”. The Frontend calls `follow(target_user_id, session_cookie)`.

FollowController: 1. Checks `follow.findUnique({ follower_id: user_id, following_id: target_user_id })`. 2. If already following: returns **400 Already following**. Flow terminates. 3. Calls `follow.create({ follower_id: user_id, following_id: target_user_id })`. 4. Calls NotificationService to create a notification for the target user: `{ type: FOLLOW, title: "New Follower", message: "@user started following you" }` 5. Returns `{ message: "Followed" }`. The Frontend changes the button to “Following”.

10.3.3 Part 3 - Unfollow

The user clicks “Unfollow”. The Frontend calls `unfollow(following_id, session_cookie)`.

FollowController: 1. Checks `follow.findUnique({ follower_id: user_id, following_id: target_user_id })`. 2. If not following: returns **404 Follow not found**. 3. Calls `follow.delete({ id })`. 4. Returns `{ message: "Unfollowed" }`. The Frontend reverts the button to “Follow”.

Note: Unfollowing does NOT create a notification for the target user.

10.4 Error Paths

Condition	HTTP Response
Already following	400 Already following
Follow record not found during unfollow	404 Not Found

10.5 Key Implementation Details

- The Follow table has a unique constraint on (`follower_id`, `following_id`), preventing duplicate follow records at the database level.
- Follow notifications use `type = FOLLOW` as a string convention. The `data` JSON field on the notification can carry the follower's user ID for the frontend to render a profile link.
- The `is_following` flag returned by `get_follow_stats` is derived by checking if a Follow record exists where `follower_id = current_user` AND `following_id = target`. This is computed server-side.

11 Sequence Diagram - Google OAuth Login

UML Type: Sequence Diagram **Category:** Auth **Source:** docs/diagrams/sequence/auth/sequence_google_oauth

11.1 Overview

This diagram describes the complete message flow for Google OAuth authentication using Google One Tap or the standard OAuth prompt. It handles three distinct cases: returning Google user, existing local user linking their Google account, and brand new users.

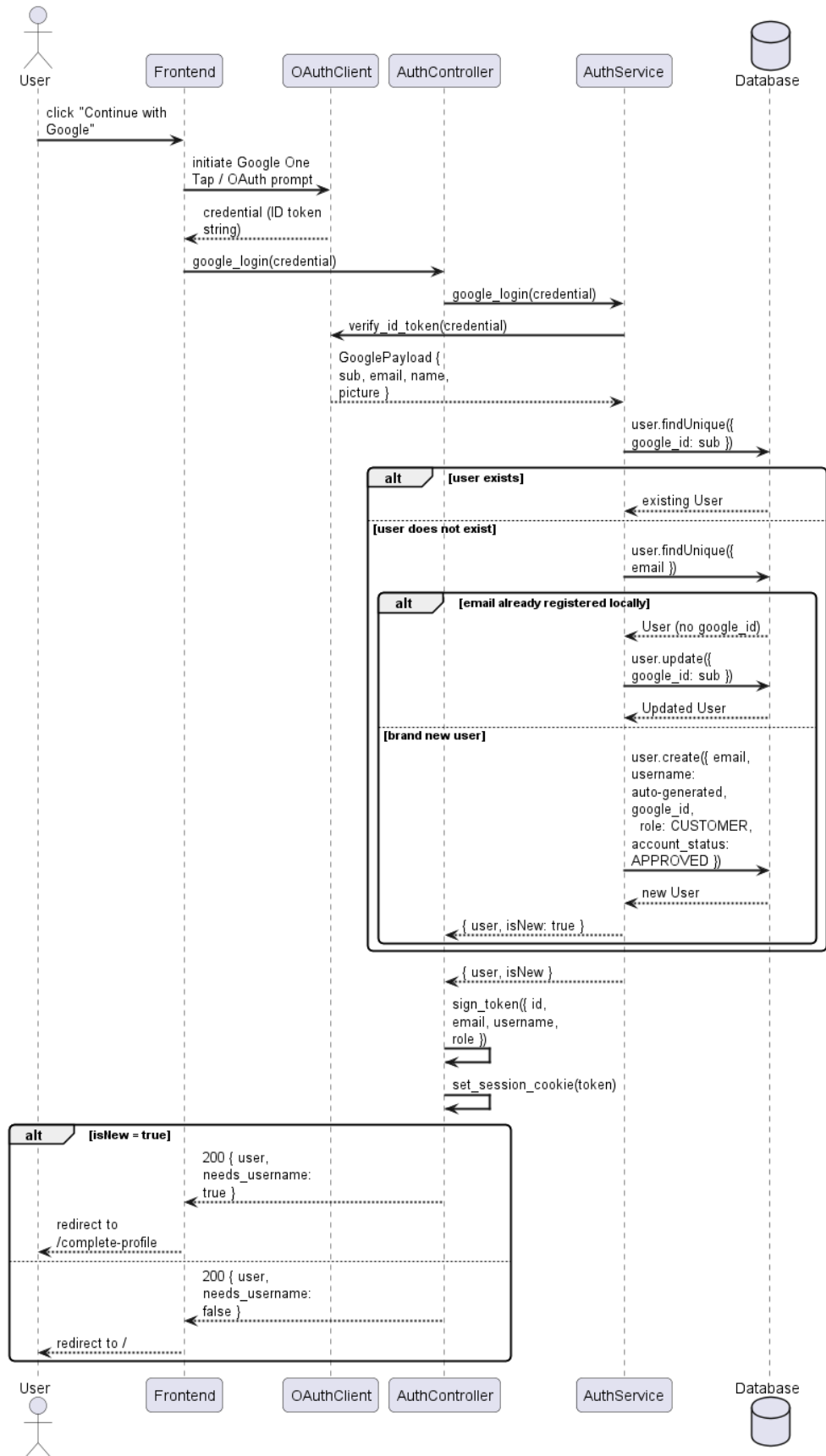


Figure 11: Sequence Diagram - Google OAuth Login

11.2 Participants

Participant	Role
User	The human actor clicking “Continue with Google”
Frontend	The Next.js client initiating the OAuth flow
OAuthClient	The Google OAuth2 client library
AuthController	The Express route handler for <code>/auth/google</code>
AuthService	Business logic for Google authentication
Database	PostgreSQL via Prisma

11.3 Flow

11.3.1 1. OAuth Prompt

The user clicks “Continue with Google”. The Frontend triggers the Google One Tap prompt (or OAuth redirect). Google’s library returns a credential string (an ID token).

11.3.2 2. Token Delivery

The Frontend sends the credential to AuthController via `google_login(credential)`.

11.3.3 3. Token Verification

AuthService sends the credential to Google’s OAuth2 API for verification. Google returns a `GooglePayload` containing `{ sub, email, name, picture }`.

11.3.4 4. User Lookup - Three Cases

AuthService queries the database to find an existing user by `google_id = sub`.

Case A - Returning Google user: A user with that `google_id` exists. AuthService uses them directly.

Case B - Existing local user (account linking): No user found by `google_id`, but a user exists with the same email. AuthService updates that user’s record, setting `google_id = sub`. This links the Google account to the existing local account.

Case C - Brand new user: No user found by either `google_id` or email. AuthService creates a new user record with an auto-generated username, sets `role = CUSTOMER`, `account_status = APPROVED`, and marks `isNew = true`.

11.3.5 5. JWT and Cookie

AuthController signs a JWT and sets the `3dex_session` cookie, same as local login.

11.3.6 6. Conditional Redirect

- If `isNew = true`: AuthController responds with `{ user, needs_username: true }`. The Frontend redirects to `/complete-profile` so the user can set a proper username.
 - Otherwise: responds with `{ user, needs_username: false }` and Frontend redirects to home.
-

11.4 Error Paths

Condition	HTTP Response
Missing credential in request body	400 Bad Request
Google token verification fails	401 Google Auth Failed

11.5 Key Implementation Details

- Auto-generated usernames for new Google users are temporary placeholders. The `/complete-profile` step forces the user to choose a valid username before proceeding.
- Google OAuth users bypass the `PENDING` account status; they are auto-approved as `CUSTOMER` immediately. Only Artist and Provider applications require admin approval.
- The `google_id` field is stored as a unique key on the `User` table, enabling fast lookup on subsequent logins without needing to reverify the email.

12 Sequence Diagram - Local Login

UML Type: Sequence Diagram **Category:** Auth **Source:** docs/diagrams/sequence/auth/sequence_local_login.

12.1 Overview

This diagram describes the message flow when a user logs in using their email and password (local authentication). It covers field validation, credential verification, JWT issuance, and S3 URL signing.

12.2 Participants

Participant	Role
User	The human actor submitting the login form
Frontend	The Next.js client managing the login form and session state
AuthController	The Express route handler for <code>/auth/login</code>
AuthService	Business logic layer for authentication
Database	PostgreSQL via Prisma
StorageService	S3 pre-signing helper for avatar and banner URLs

12.3 Flow

12.3.1 1. Form Submission

The user fills in the login form with their email and password and submits it. The Frontend sends a POST request to the AuthController.

12.3.2 2. Field Validation

AuthController immediately validates that both `email` and `password` are present in the request body.

- If either field is missing: returns **400 Missing fields** to the Frontend, which displays an error to the user. The flow terminates.

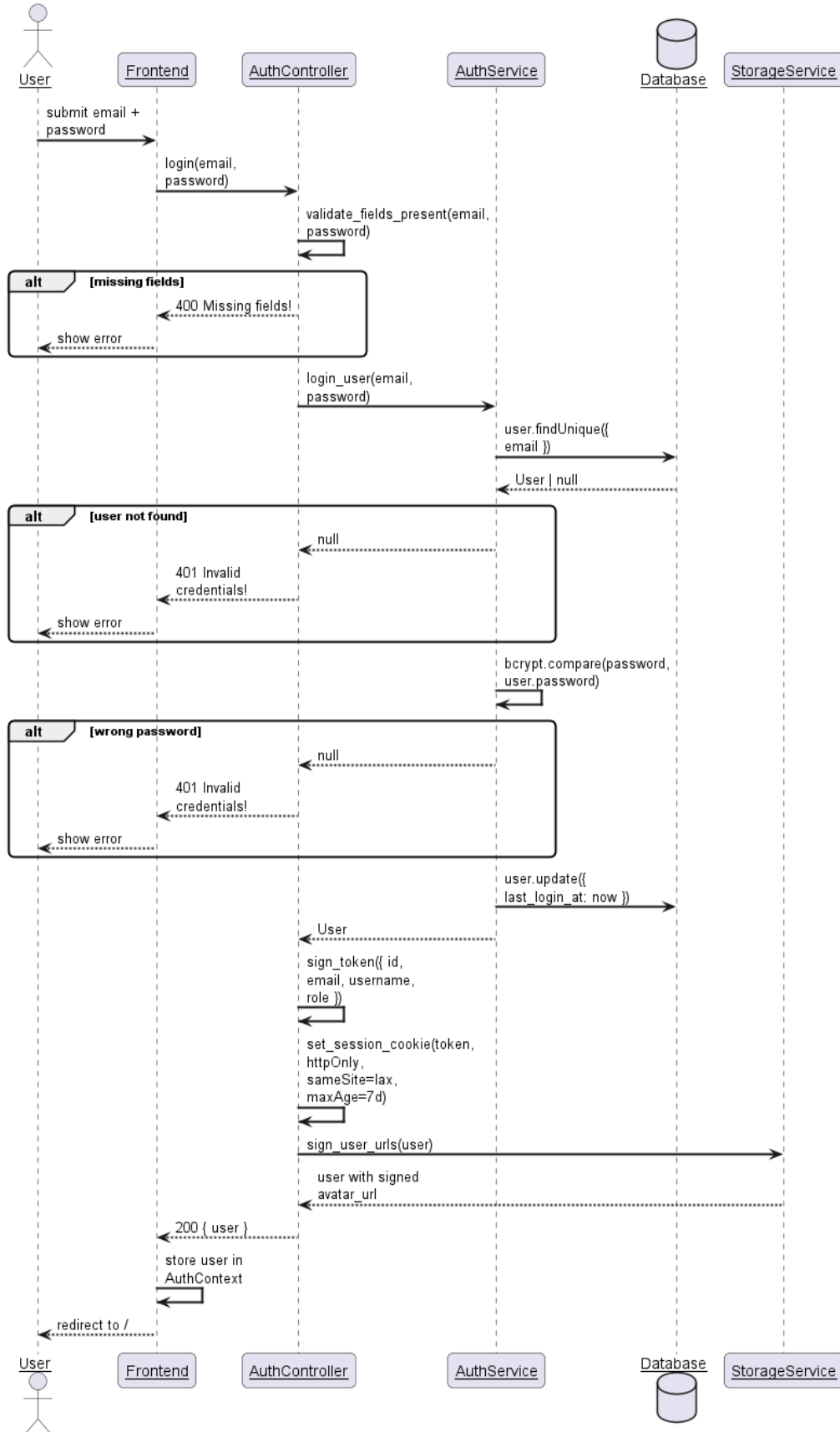


Figure 12: Sequence Diagram - Local Login

12.3.3 3. Credential Lookup

AuthController delegates to AuthService, which queries the database for a user record with the given email.

- If no user is found: AuthService returns null, AuthController returns **401 Invalid credentials**. The flow terminates.

12.3.4 4. Password Verification

AuthService calls `bcrypt.compare(password, user.password)` to verify the submitted plaintext against the stored hash.

- If the password does not match: returns **401 Invalid credentials**. The flow terminates.

12.3.5 5. Last Login Update

AuthService updates the user's `last_login_at` timestamp in the database.

12.3.6 6. JWT Signing and Cookie

AuthController signs a JWT containing `{ id, email, username, role }`. It then sets an HTTP-only session cookie named `3dex_session` with `sameSite=lax` and a 7-day `maxAge`.

12.3.7 7. S3 URL Signing

AuthController calls `StorageService` to generate a presigned S3 URL for the user's `avatar_url`. The signed URL is embedded in the response.

12.3.8 8. Response and Redirect

AuthController returns **200** with the user object (including the signed avatar URL). The Frontend stores the user in `AuthContext` and redirects to the home page.

12.4 Error Paths

Condition	HTTP Response
Missing email or password	400 Missing fields
User not found by email	401 Invalid credentials
Password mismatch	401 Invalid credentials

12.5 Key Implementation Details

- The error message is intentionally generic (“Invalid credentials”) for both user-not-found and wrong-password cases, preventing user enumeration attacks.
- The cookie is `httpOnly`, meaning JavaScript on the client cannot read it. The server reads it on every subsequent authenticated request via the auth middleware.
- `StorageService` calls MinIO (S3-compatible) to generate a short-lived presigned GET URL for the avatar so the client can display the profile image without exposing permanent keys.

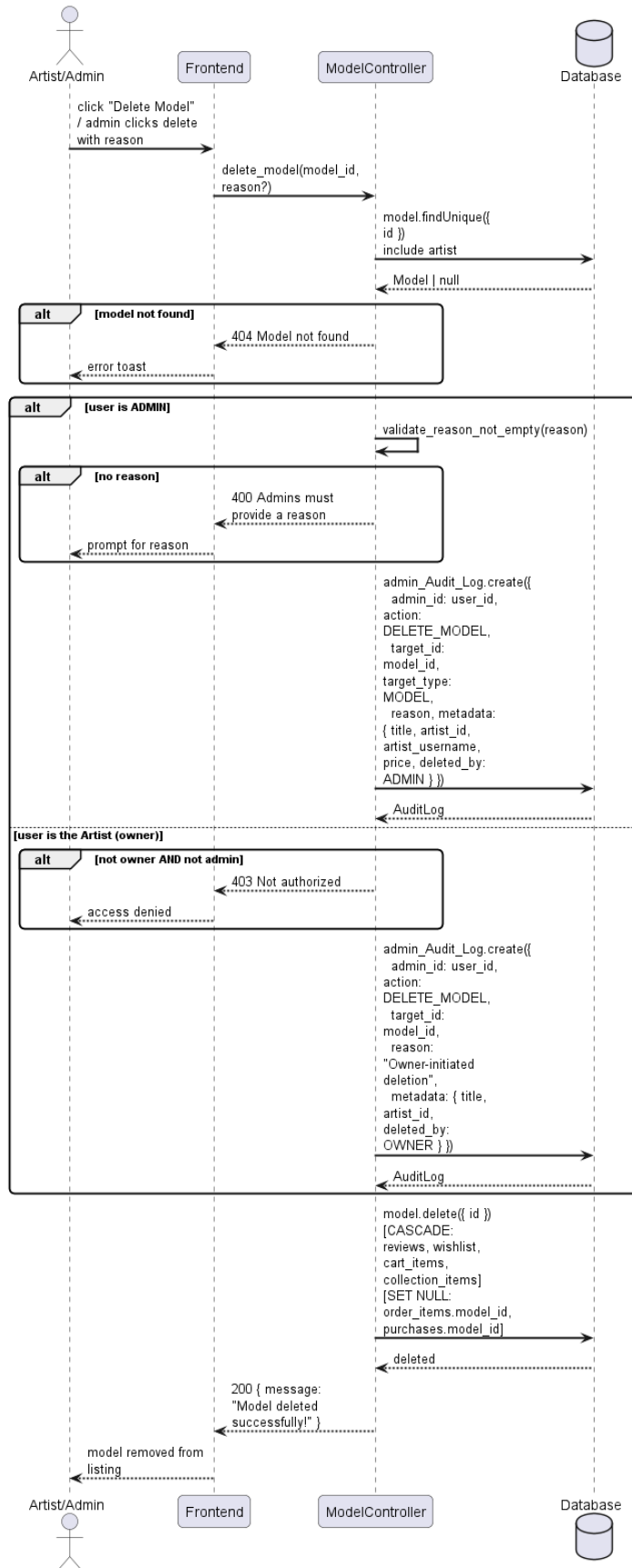


Figure 13: Sequence Diagram - Model Delete

13 Sequence Diagram - Model Delete

UML Type: Sequence Diagram Category: Catalog Source: docs/diagrams/sequence/catalog/sequence_model_delete

13.1 Overview

This diagram describes how a model is deleted, covering two distinct paths: an Artist deleting their own model, and an Admin force-deleting a model with a mandatory reason. Both paths write to the audit log.

13.2 Participants

Participant	Role
Artist / Admin	The actor initiating the deletion
Frontend	The client
ModelController	Express route handler for model deletion
Database	PostgreSQL via Prisma

13.3 Flow

13.3.1 1. Delete Request

The Artist or Admin triggers the delete action. The Frontend sends `delete_model(model_id, reason?)`.

13.3.2 2. Model Lookup

ModelController queries the database for the model, including artist details. - If not found: returns **404 Model not found**. Flow terminates.

13.3.3 3. Admin Path

If the caller has the ADMIN role: 1. ModelController validates that `reason` is present. 2. If reason is empty: returns **400 Admins must provide a reason**. Flow terminates. 3. Creates an `Admin_Audit_Log` record with: - `action = DELETE_MODEL` - `target_id = model_id` - `target_type = MODEL` - `reason = <entered reason>` - `metadata = { title, artist_id, artist_username, price, deleted_by: ADMIN }`

13.3.4 4. Artist (Owner) Path

If the caller is not an admin: 1. If `model.artist_id != user.id`: returns **403 Not authorized**. Flow terminates. 2. Creates an `Admin_Audit_Log` record with: - `action = DELETE_MODEL` - `reason = "Owner-initiated deletion"` - `metadata = { title, artist_id, deleted_by: OWNER }`

13.3.5 5. Deletion with Cascade

ModelController calls `model.delete({ id })`. The database enforces: - **CASCADE deletes**: Review, Wishlist, Cart_Item, Collection_Item - **SET NULL**: Order_Item.model_id, Purchase.model_id

This means Purchase records and Order Items are preserved (for financial record keeping) but their model reference is nullified.

13.3.6 6. Response

ModelController returns **200 Model deleted successfully**. The Frontend removes the model from the listing.

13.4 Error Paths

Condition	HTTP Response
Model not found	404 Not Found
Admin with no reason	400 Bad Request
Not owner and not admin	403 Forbidden

13.5 Key Implementation Details

- Owner-initiated deletions also write an audit log, treating the owner as the “admin” of their own content. This is notable because the `Admin_Audit_Log` table records these self-deletions with `admin_id = artist_id`.
- The SET NULL behavior on `Purchase.model_id` and `Order_Item.model_id` is critical: it ensures that a customer’s purchase history remains intact even after the underlying model is removed, preserving financial records.
- CASCADE deletions on reviews mean ratings are lost when a model is deleted. This is expected.

14 Sequence Diagram - Model Detail

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_model_d

14.1 Overview

This diagram shows what happens when a user clicks on a model card to view its detail page. It covers ownership checks, URL signing, and conditional rendering based on whether the user has purchased the model.

14.2 Participants

Participant	Role
User	Any visitor (authenticated or guest)
Frontend	The Next.js client rendering the model detail page
ModelController	Express route handler for model detail
PurchaseService	Business logic for checking purchase status
StorageService	S3 URL presigner
Database	PostgreSQL via Prisma

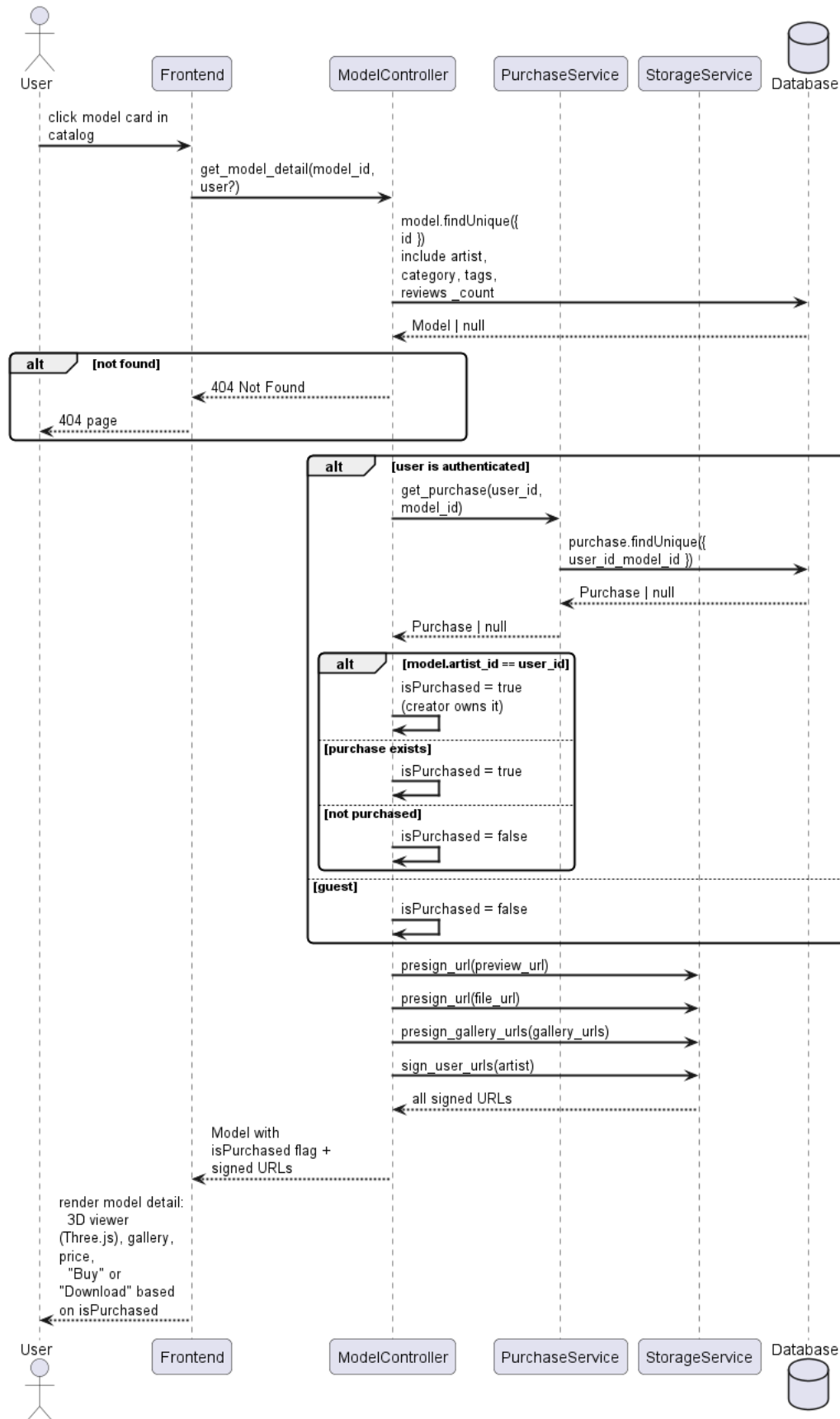


Figure 14: Sequence Diagram - Model Detail

14.3 Flow

14.3.1 1. Model Lookup

The Frontend calls `get_model_detail(model_id, user?)`. `ModelController` queries the database for the model including its `artist`, `category`, `tags`, and `reviews._count`.

- If not found: returns **404 Not Found**. The Frontend shows a 404 page. Flow terminates.

14.3.2 2. Purchase Check (Authenticated Users Only)

If the user is authenticated:

1. `ModelController` calls `PurchaseService` to check `purchase.findUnique({ user_id_model_id })`.
2. Three sub-cases determine `isPurchased`:
 - If `model.artist_id == user_id`: the artist always owns their own model, so `isPurchased = true`.
 - If a `Purchase` record exists: `isPurchased = true`.
 - Otherwise: `isPurchased = false`.

If the user is a guest: `isPurchased = false`.

14.3.3 3. URL Signing

`ModelController` calls `StorageService` to presign: - The model's `preview_url` - The model's `file_url` - All `gallery_urls` - The artist's `avatar_url`

14.3.4 4. Response

`ModelController` returns the model object with the `isPurchased` flag and all signed URLs. The Frontend conditionally renders either a “Buy” button (not purchased) or a “Download” button (purchased), alongside the Three.js 3D viewer, gallery, and price information.

14.4 Key Implementation Details

- The `isPurchased` flag is derived server-side, not client-side, to prevent tampering.
- Even non-purchased users can view the model's 3D preview (using the presigned `preview_url`), but the actual downloadable file requires purchase authorization.
- The creator's `file_url` presigned URL is returned to all callers. The download endpoint enforces the purchase gate; the detail page URL itself does not block access to the signed URL but the actual file is only meaningful after download authorization is confirmed separately.
- Gallery images are also presigned, meaning they are only accessible via time-limited URLs returned from the API.

15 Sequence Diagram - Model Download

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_model_

15.1 Overview

This diagram describes the secure file download flow. Download access is controlled server-side, requiring proof of purchase (or special authorization) before a short-lived presigned download URL is generated.

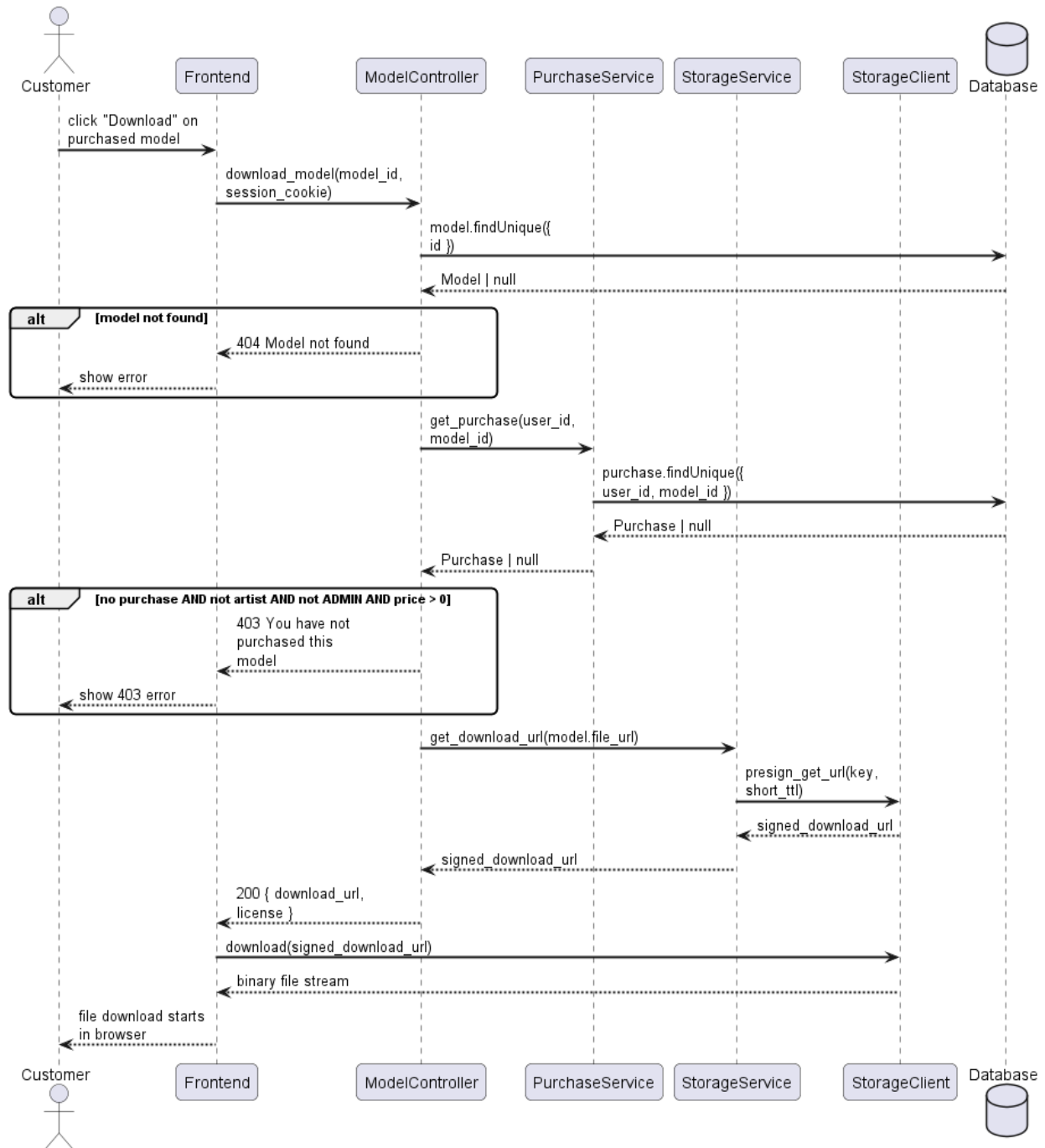


Figure 15: Sequence Diagram - Model Download

15.2 Participants

Participant	Role
Customer	The authenticated user requesting the download
Frontend	The client
ModelController	Express route handler for download
PurchaseService	Purchase verification logic
StorageService	S3 URL presigner
StorageClient (S3)	MinIO backend
Database	PostgreSQL via Prisma

15.3 Flow

15.3.1 1. Download Request

The Customer clicks “Download” on a purchased model. The Frontend calls `download_model(model_id, session_cookie)`.

15.3.2 2. Model Lookup

ModelController queries the database for the model by ID. - If not found: returns **404 Model not found**. Flow terminates.

15.3.3 3. Purchase Verification

ModelController calls PurchaseService to check `purchase.findUnique({ user_id, model_id })`.

Access is granted if any of the following is true: - A **Purchase** record exists for the user - The user is the model’s artist (they own their own work) - The user has the **ADMIN** role - The model’s price is 0 (free)

If none of these conditions are met: returns **403 You have not purchased this model**. Flow terminates.

15.3.4 4. Presigned Download URL

ModelController calls StorageService, which calls MinIO to generate a presigned GET URL with a short time-to-live (short TTL). This ensures the link expires quickly and cannot be shared indefinitely.

15.3.5 5. Download Initiation

ModelController returns **200 { download_url, license }**. The Frontend uses the signed URL to initiate a direct browser download from MinIO. The file stream is delivered directly from S3 to the user’s browser.

15.4 Error Paths

Condition	HTTP Response
Model not found	404 Model not found
No purchase AND not artist AND not admin AND price > 0	403 Forbidden

15.5 Key Implementation Details

- The `license` field is returned alongside the download URL so the client can display a reminder of the license terms (Personal Use or Commercial Use).
- The short TTL presigned URL prevents unauthorized redistribution of download links. Even if a link is shared, it expires after a brief window.
- Free models (`price = 0`) bypass the purchase check entirely, as their file is available to anyone.
- The admin bypass allows admins to download any model for moderation review purposes.

16 Sequence Diagram - Model Review

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_model_r

16.1 Overview

This diagram covers the flow for a customer submitting a star rating and comment review for a 3D model. A prior purchase is required, and the review automatically updates the model's aggregate rating.

16.2 Participants

Participant	Role
Customer	The authenticated user writing the review
Frontend	The client
ReviewController	Express route handler for model reviews
Database	PostgreSQL via Prisma

16.3 Flow

16.3.1 1. Review Form

The Customer clicks "Write a Review" on a model they own. The Frontend shows a form with a 1-5 star selector and an optional comment field.

16.3.2 2. Submission

The Customer submits `{ rating: 4, comment: "Great model!" }`. The Frontend calls `create_review(model_id, rating, comment, session_cookie)`.

16.3.3 3. Rating Validation

ReviewController validates that `rating` is between 1 and 5 (inclusive). - If invalid: returns **400 Bad Request**. Flow terminates.

16.3.4 4. Purchase Verification

ReviewController queries `purchase.findUnique({ user_id_model_id: { user_id, model_id } })`. - If no purchase record: returns **403 You must buy the model first**. Flow terminates.

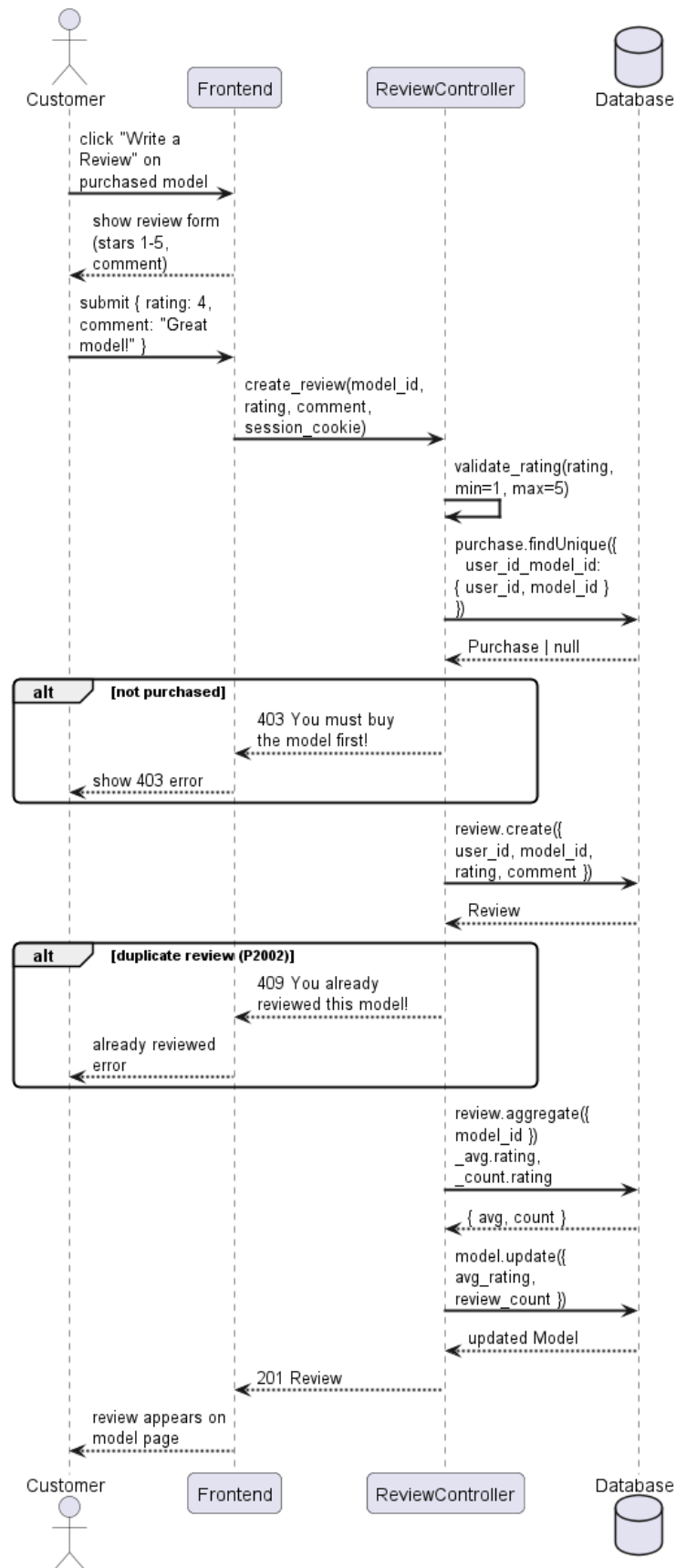


Figure 16: Sequence Diagram - Model Review

16.3.5 5. Review Creation

ReviewController calls `review.create({ user_id, model_id, rating, comment })`.

If a review already exists for this user-model pair (Prisma error P2002): - Returns **409 You already reviewed this model**. Flow terminates.

16.3.6 6. Rating Recalculation

ReviewController calls `review.aggregate({ model_id })` to compute `_avg.rating` and `_count.rating`. It then calls `model.update({ avg_rating, review_count })` to update the denormalized fields.

16.3.7 7. Response

ReviewController returns **201 Created**. The review appears on the model detail page.

16.4 Error Paths

Condition	HTTP Response
Rating outside 1-5 range	400 Bad Request
No purchase record	403 Forbidden
Duplicate review	409 Conflict

16.5 Key Implementation Details

- The purchase gate prevents review spam from users who have not engaged with the model.
- The unique constraint (`user_id, model_id`) on the `Review` table enforces one review per user per model at the database level.
- After each review, the `avg_rating` and `review_count` on the `Model` record are updated immediately (in the same request). These are denormalized fields kept in sync to avoid expensive aggregation queries on every model list fetch.

17 Sequence Diagram - Model Update

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_model_u

17.1 Overview

This diagram describes the flow for an Artist editing their existing model's metadata (title, price, tags, category, license, etc.). File replacement is not covered here; this is metadata-only editing.

17.2 Participants

Participant	Role
Artist	The authenticated artist who owns the model
Frontend	The client

Participant	Role
ModelController	Express route handler for model updates
ModelService	Business logic for model persistence
Database	PostgreSQL via Prisma

17.3 Flow

17.3.1 1. Edit Form Load

The Artist opens the model edit form. The Frontend calls `get_model_detail(model_id)` to pre-fill the form with current values.

17.3.2 2. Changes and Submission

The Artist modifies fields (title, price, tags, category, etc.) and submits. The Frontend calls `update_model(model_id, { title, price, category, tags, license, is_printable, is_nsfw })`.

17.3.3 3. Authorization Check

ModelController fetches the model and validates: - If not found: returns **404 Not Found**. - If `model.artist_id != user.id` AND the user is not an ADMIN: returns **403 Not authorized**.

17.3.4 4. Update Execution

ModelController delegates to ModelService, which: 1. If the category has changed, resolves the new `category_id` from the provided slug via `category.findUnique`. 2. Calls `model.update` with safe fields: `title, price, category_id, license, is_printable, is_nsfw`. 3. For tags: first disconnects all existing tags (`set: []`), then connects the new set of tags.

17.3.5 5. Response

ModelService returns the updated model. ModelController returns **200** with the updated model. The Frontend reflects the changes.

17.4 Error Paths

Condition	HTTP Response
Model not found	404 Not Found
Not the owner and not admin	403 Not Authorized

17.5 Key Implementation Details

- The tags update uses a `set: []` then `connect: [...]` pattern instead of a diff, meaning the entire tag set is replaced on each edit.
- `file_url` and `preview_url` are not editable through this endpoint. File replacement would require a separate presigned PUT URL flow similar to the initial upload.

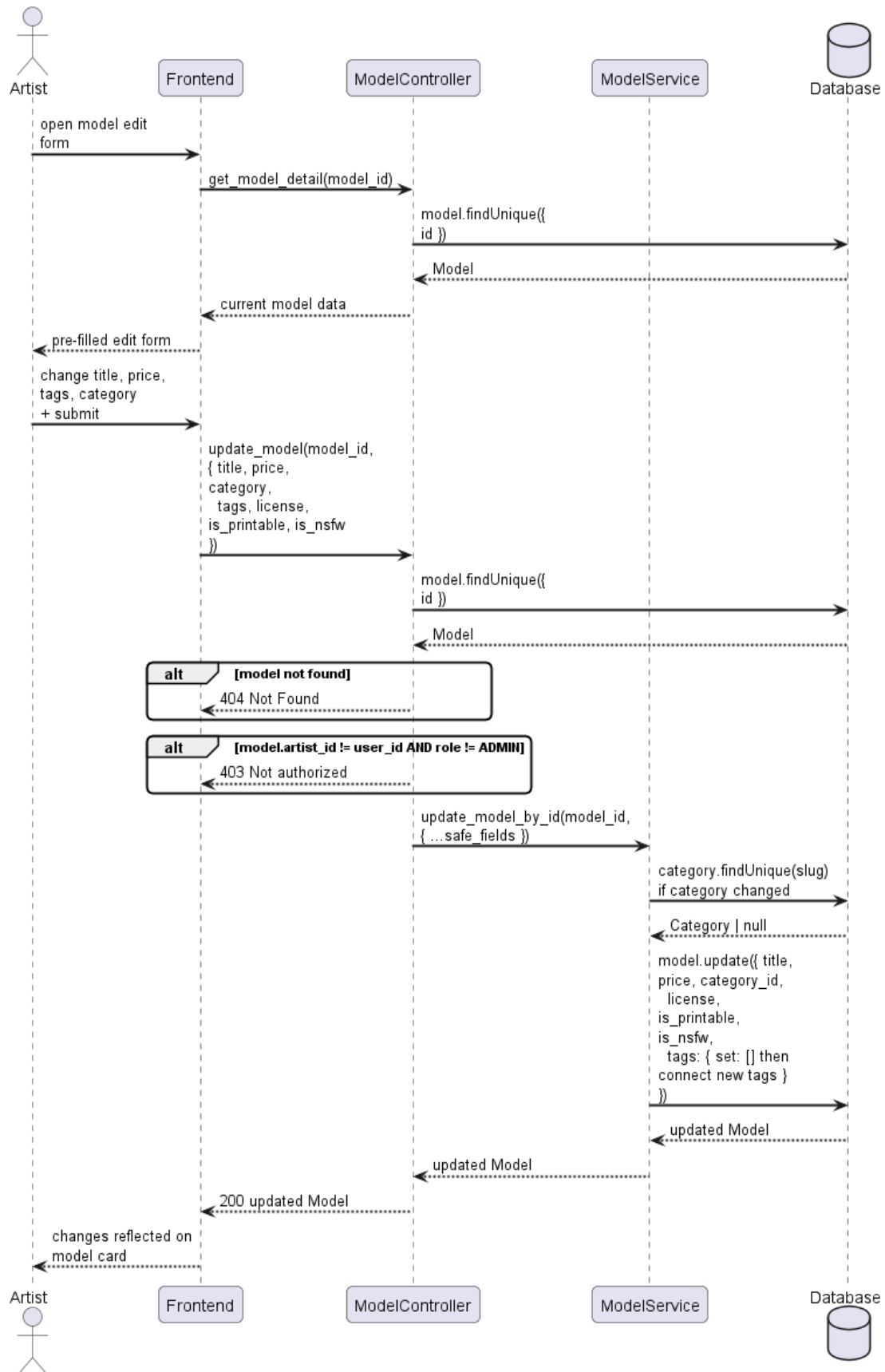


Figure 17: Sequence Diagram - Model Update

- Admins can also update any model's metadata, which is why the authorization check includes `role == ADMIN` as a bypass condition.
- Updating a model does not reset its `status` back to `PENDING`. If an artist edits a model that was already approved, it stays approved. Policy enforcement of re-review after edit is not currently implemented.

18 Sequence Diagram - Model Upload

UML Type: Sequence Diagram **Category:** Catalog **Source:** docs/diagrams/sequence/catalog/sequence_model_upload

18.1 Overview

This diagram covers the complete model upload workflow. Files are uploaded directly to S3 using pre-signed PUT URLs, then the model metadata is saved to the database. Finally, an embedding is generated asynchronously for semantic search.

18.2 Participants

Participant	Role
Artist	The authenticated artist user
Frontend	The Next.js client
StorageController	Express route handler for presigned URL generation
StorageService	S3 URL presigner
StorageClient (S3)	MinIO S3 storage backend
ModelController	Express route handler for model creation
ModelService	Business logic for model persistence
EmbeddingService	Semantic embedding generator
Database	PostgreSQL via Prisma

18.3 Flow

18.3.1 1. Presigned URL for GLB File

The Artist fills the upload form. The Frontend requests a presigned PUT URL for the `.glb` file by calling `get_upload_url(filename, content_type: model/gltf-binary)`.

StorageController delegates to StorageService, which calls MinIO to generate a `presigned_url` and an S3 `key`. The Frontend receives both.

18.3.2 2. Direct Upload to S3

The Frontend uploads the raw GLB binary directly to MinIO using the presigned URL (`PUT presigned_url`). The file never passes through the application server.

18.3.3 3. Presigned URL for Preview Image

The process repeats for the WebP preview image. The Frontend receives a separate `upload_url` and `preview_key`.

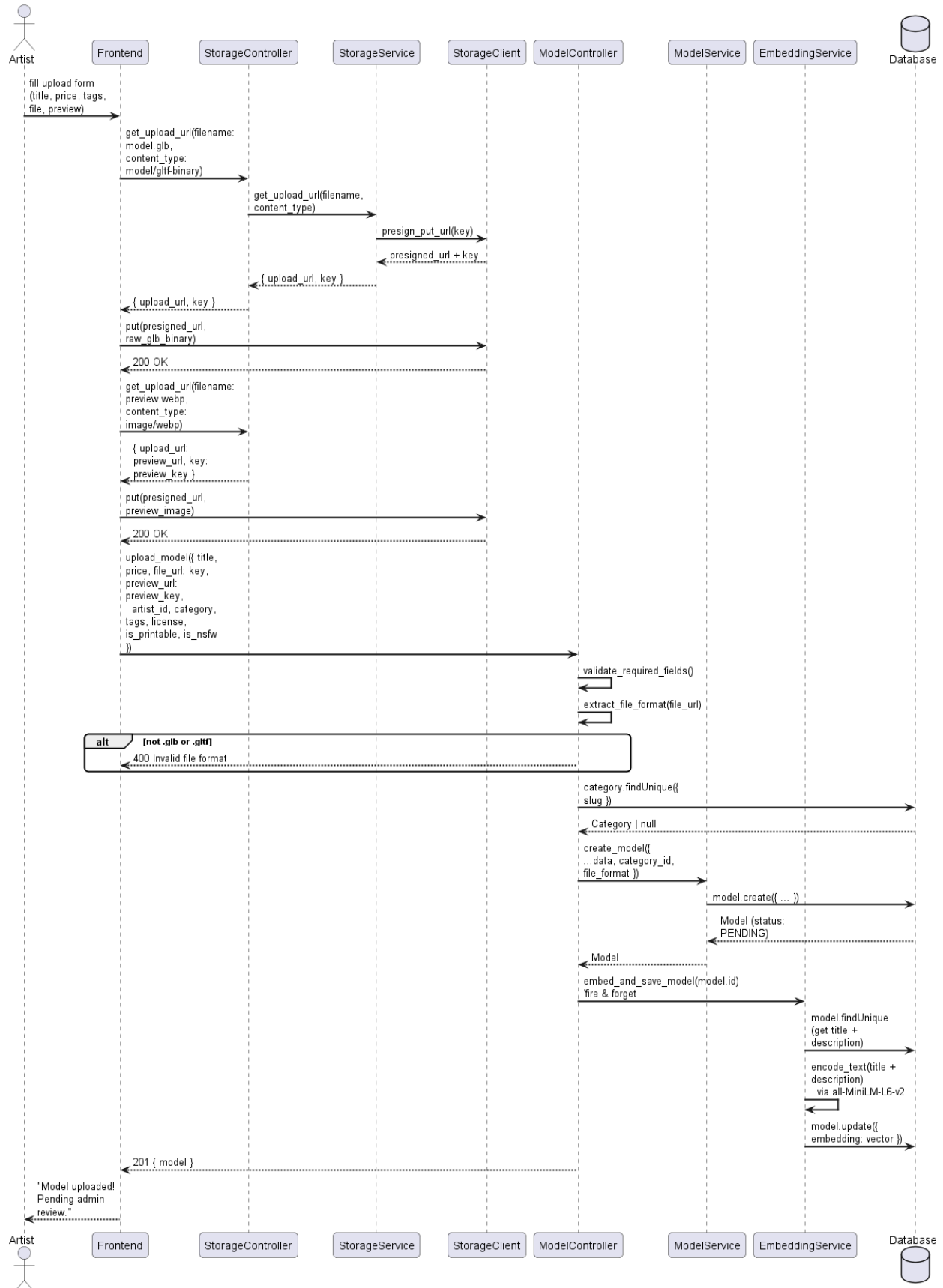


Figure 18: Sequence Diagram - Model Upload

18.3.4 4. Preview Upload

The Frontend uploads the preview image directly to MinIO.

18.3.5 5. Model Metadata Submission

The Frontend submits the model metadata to ModelController:

```
{ title, price, file_url: key, preview_url: preview_key,
  artist_id, category, tags, license, is_printable, is_nsfw }
```

ModelController: 1. Validates required fields are present. 2. Extracts the file extension from `file_url` to populate `file_format`. 3. If the format is not `.glb` or `.gltf`: returns **400 Invalid file format**. 4. Resolves `category_id` from the category slug via `category.findUnique({ slug })`. 5. Delegates to ModelService, which calls `model.create(...)`.

The model is created with `status = PENDING`.

18.3.6 6. Embedding Generation (Fire and Forget)

ModelController immediately calls `EmbeddingService.embed_and_save_model(model.id)` without awaiting the result. This is a background task.

EmbeddingService: 1. Fetches the model's `title` and `description`. 2. Encodes the combined text using the `all-MiniLM-L6-v2` sentence embedding model. 3. Saves the resulting 384-dimensional vector to `model.embedding` via Prisma.

18.3.7 7. Response

ModelController returns **201 Created** with the model object while the embedding runs in the background. The Frontend shows the artist a “Model uploaded! Pending admin review.” message.

18.4 Error Paths

Condition	HTTP Response
Missing required metadata fields	400 Bad Request
File format not GLB or GLTF	400 Invalid file format
Category slug not found	404 or null <code>category_id</code>

18.5 Key Implementation Details

- The direct-to-S3 upload pattern means the application server never handles the file bytes, keeping it lightweight.
- New models start as `PENDING`. They will not appear in the public catalog until an admin approves them.
- The embedding is generated fire-and-forget; upload success does not depend on embedding completion.
- The `all-MiniLM-L6-v2` model produces 384-dimension vectors stored in the `embedding` column using PostgreSQL's `pgvector` extension.

19 Sequence Diagram - Notifications

UML Type: Sequence Diagram **Category:** Notifications **Source:** docs/diagrams/sequence/notifications/sequence

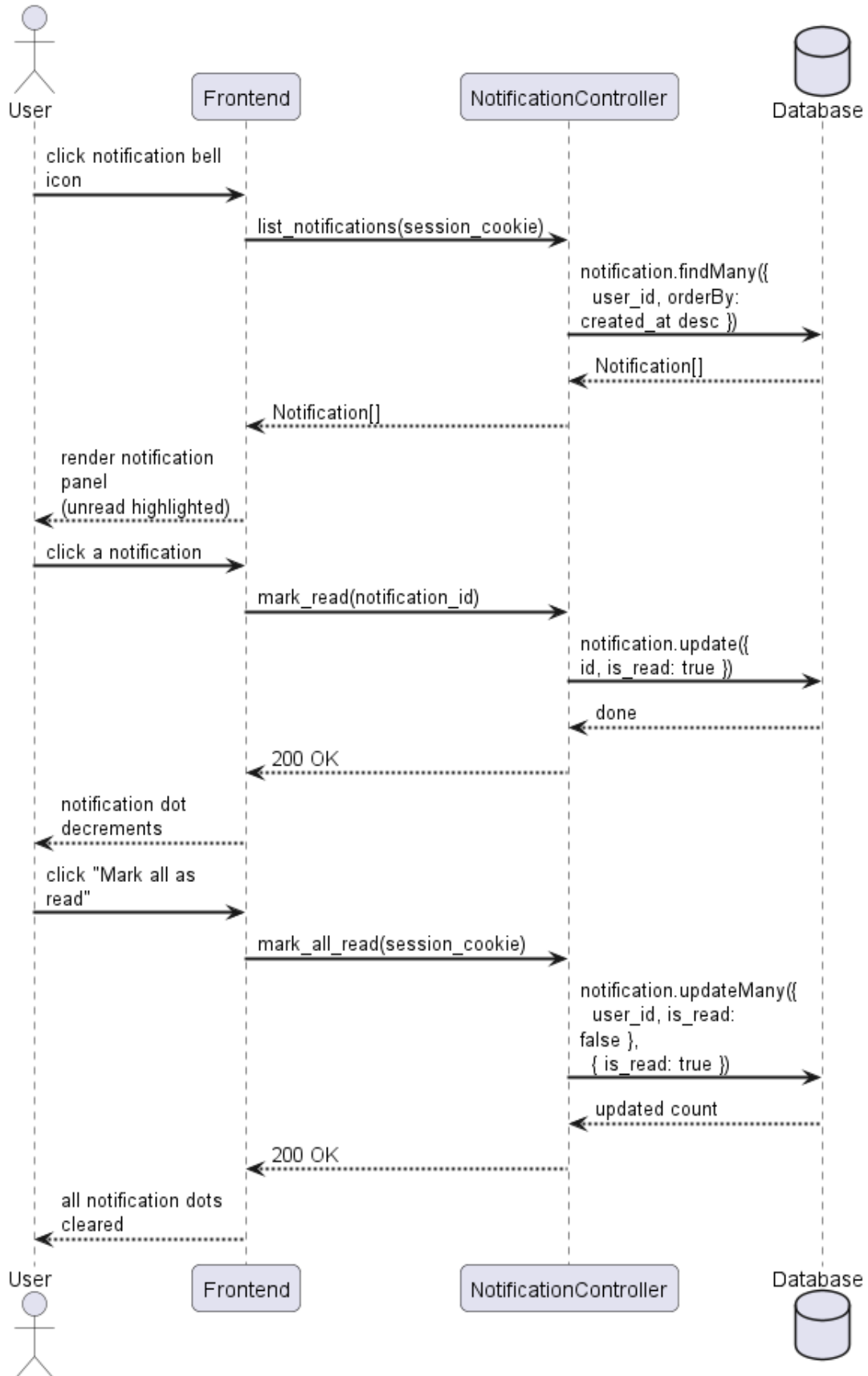


Figure 19: Sequence Diagram - Notifications

19.1 Overview

This diagram covers the notification panel lifecycle: loading notifications, marking a single notification as read, and marking all notifications as read at once.

19.2 Participants

Participant	Role
User	The authenticated user
Frontend	The client
NotificationController	Express route handler for notification operations
Database	PostgreSQL via Prisma

19.3 Flow

19.3.1 Part 1 - Load Notifications

The user clicks the notification bell icon. The Frontend calls `list_notifications(session_cookie)`.

NotificationController queries `notification.findMany({ user_id, orderBy: created_at desc })`. Returns all notifications for the user. The Frontend renders the panel with unread notifications highlighted.

19.3.2 Part 2 - Mark Single as Read

The user clicks on a specific notification. The Frontend calls `mark_read(notification_id)`.

NotificationController calls `notification.update({ id, is_read: true })`. Returns **200 OK**. The Frontend decrements the unread badge count.

19.3.3 Part 3 - Mark All as Read

The user clicks “Mark all as read”. The Frontend calls `mark_all_read(session_cookie)`.

NotificationController calls `notification.updateMany({ user_id, is_read: false }, { is_read: true })`. Returns **200 OK**. The Frontend clears all unread dots.

19.4 Key Implementation Details

- Notifications are created by `NotificationService` at various event points: payment completion, follow events, and print job status changes.
- The `type` field on `Notification` is a free-form string used by the frontend to determine rendering style (icon, color) and navigation destination (via the `data` field).
- There is no real-time push mechanism (WebSocket or SSE) shown here. Notifications are pulled on demand when the user opens the bell panel.
- The `data` JSON field can carry arbitrary metadata (e.g., model ID, order ID) enabling the frontend to deep-link to relevant content when the notification is clicked.

20 Sequence Diagram - Payment Webhook

UML Type: Sequence Diagram **Category:** Commerce **Source:** docs/diagrams/sequence/commerce/sequence_pay

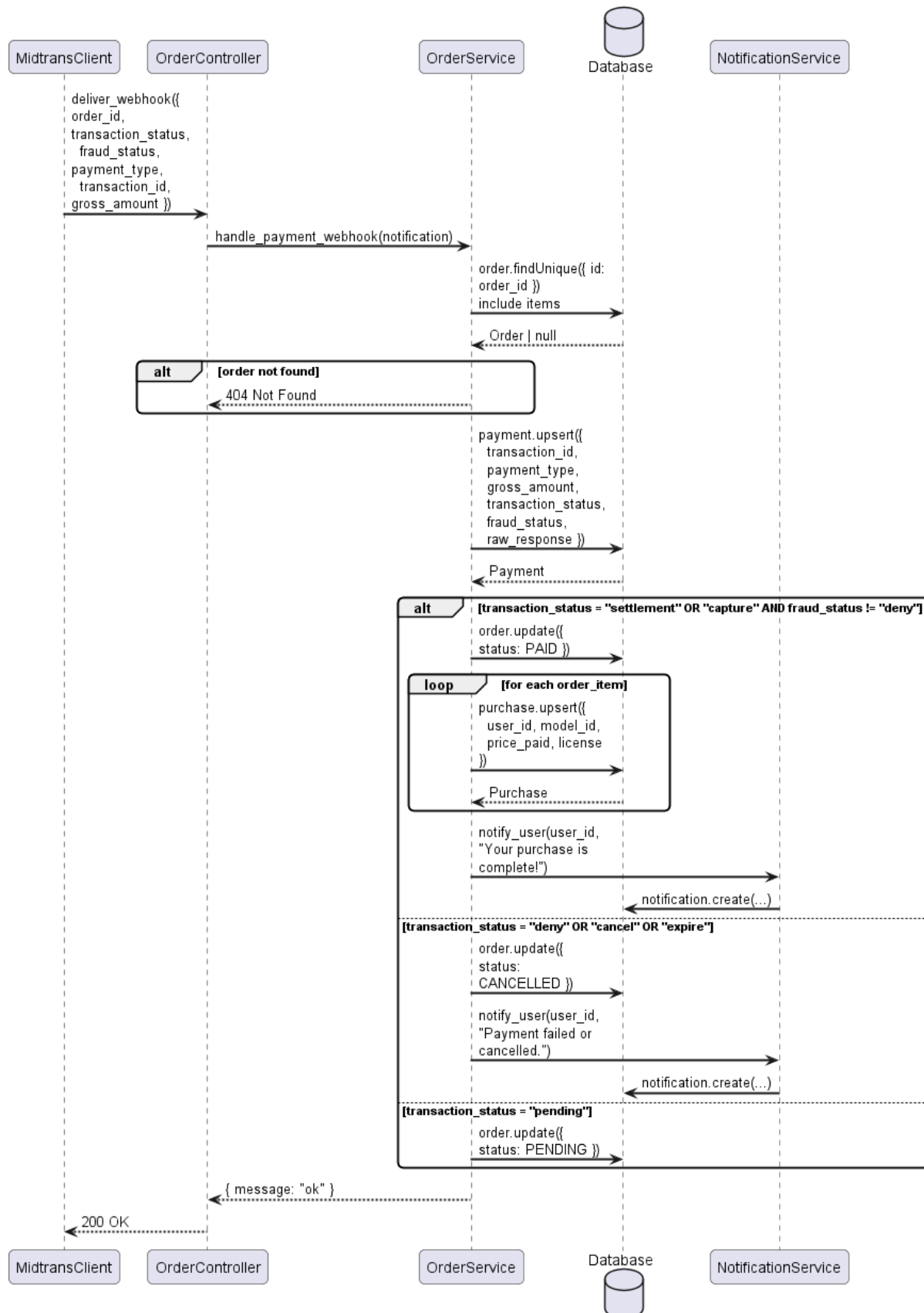


Figure 20: Sequence Diagram - Payment Webhook

20.1 Overview

This diagram shows what happens when Midtrans delivers a payment notification webhook to the system. This is the critical step where a PENDING order becomes PAID and the user's purchased models are unlocked.

20.2 Participants

Participant	Role
MidtransClient	The Midtrans payment gateway delivering the notification
OrderController	Express route handler for <code>/orders/notification</code>
OrderService	Business logic for payment processing
Database	PostgreSQL via Prisma
NotificationService	In-app notification creator

20.3 Flow

20.3.1 1. Webhook Delivery

Midtrans sends a POST to `/orders/notification` with payload:

```
{
  order_id, transaction_status, fraud_status,
  payment_type, transaction_id, gross_amount
}
```

20.3.2 2. Order Lookup

OrderService queries `order.findUnique({ id: order_id })` including its items. - If not found: returns **404**. Midtrans may retry. Flow terminates.

20.3.3 3. Payment Record Upsert

OrderService upserts a `Payment` record using `transaction_id` as the unique key:

```
payment.upsert({
  transaction_id, payment_type, gross_amount,
  transaction_status, fraud_status, raw_response
})
```

This ensures idempotency: if Midtrans delivers the same webhook twice, the payment record is updated, not duplicated.

20.3.4 4. Status Branching

Path A - Payment Success:

If `transaction_status = "settlement"` OR `(transaction_status = "capture" AND fraud_status != "deny")`:

- `order.update({ status: PAID })`
- For each `Order_Item`: `purchase.upsert({ user_id, model_id, price_paid, license })`
- `NotificationService.notify_user(user_id, "Your purchase is complete!")`

Path B - Payment Failed or Cancelled:

If `transaction_status` IN ["deny", "cancel", "expire"]:

1. `order.update({ status: CANCELLED })`
2. `NotificationService.notify_user(user_id, "Payment failed or cancelled.")`

Path C - Payment Pending:

If `transaction_status` = "pending": 1. `order.update({ status: PENDING })` (no change in most cases)

20.3.5 5. Response

`OrderService` returns `{ message: "ok" }`. `OrderController` responds **200 OK** to Midtrans.

20.4 Key Implementation Details

- The webhook endpoint must return 200 promptly. Midtrans interprets timeouts or non-200 responses as failures and retries.
- The `Purchase.upsert` pattern handles retried webhooks gracefully: if a purchase already exists for (`user_id`, `model_id`), no duplicate is created.
- The `raw_response` JSON field on `Payment` stores the full Midtrans webhook payload for debugging and audit purposes.
- Notification creation is done inline in the webhook handler. If this fails, the order status has already been updated, so the notification is best-effort.

21 Sequence Diagram - Print Job Management

UML Type: Sequence Diagram **Category:** Print **Source:** docs/diagrams/sequence/print/sequence_print_job_m

21.1 Overview

This diagram describes the provider-side view of managing incoming print jobs. It covers viewing jobs, accepting a job, shipping with tracking information, and marking delivery as complete.

21.2 Participants

Participant	Role
Provider	The authenticated printing service provider
Frontend	The client
PrintController	Express route handler for print job management
Database	PostgreSQL via Prisma

21.3 Flow

21.3.1 Part 1 - View Incoming Jobs

The Provider navigates to their job dashboard. The Frontend calls `get_provider_jobs(session_cookie)`.

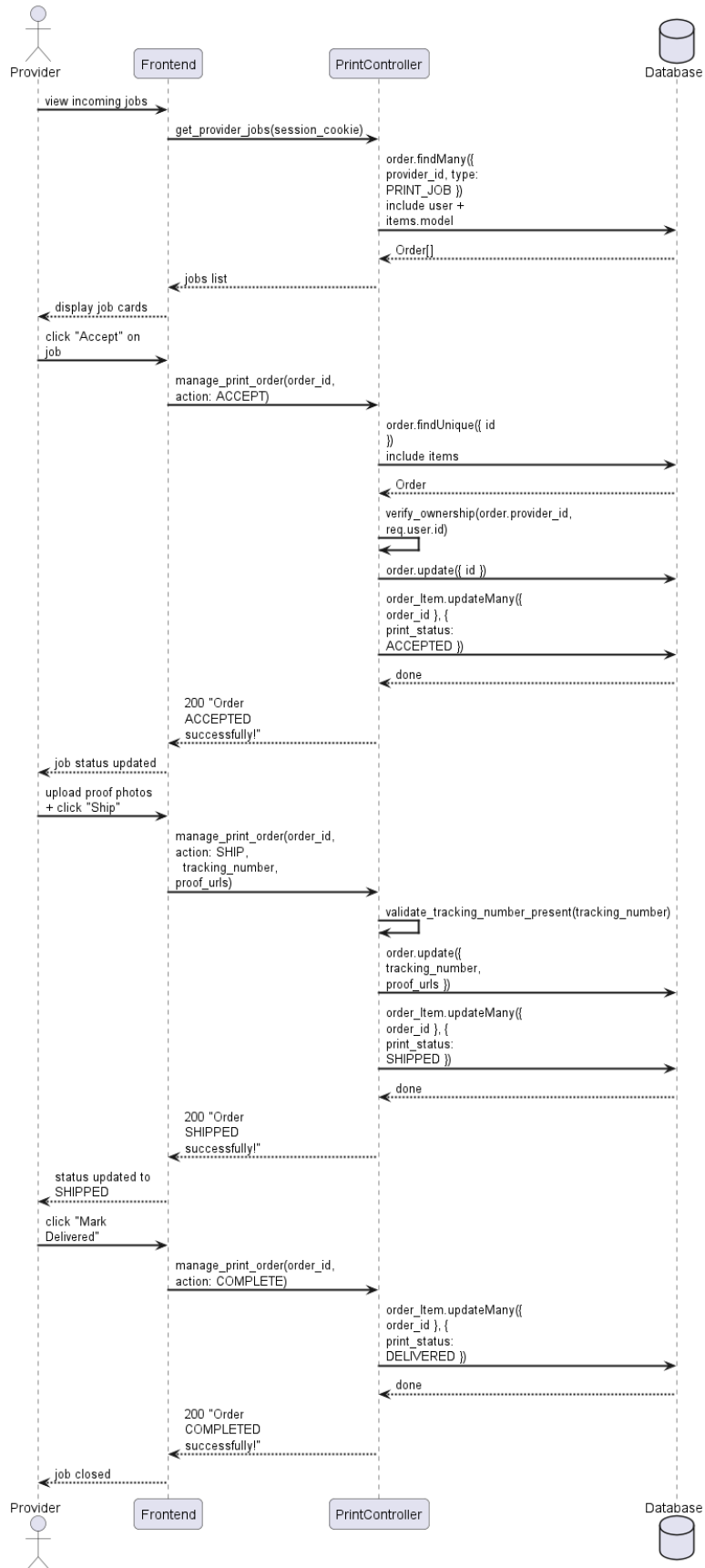


Figure 21: Sequence Diagram - Print Job Management

PrintController queries `order.findMany({ provider_id, type: PRINT_JOB })` including the customer user and all order items with their models. Returns the job list. The Frontend displays job cards.

21.3.2 Part 2 - Accept a Job

The Provider clicks “Accept” on a job. The Frontend calls `manage_print_order(order_id, action: ACCEPT)`.

PrintController: 1. Fetches the order and verifies that `order.provider_id == req.user.id` (ownership check). 2. Updates the order record. 3. Updates all `Order_Item` records for this order: `print_status = ACCEPTED`.

Returns **200 “Order ACCEPTED successfully!”**. The Frontend updates the job status.

21.3.3 Part 3 - Ship the Job

The Provider uploads proof photos and enters a tracking number, then clicks “Ship”. The Frontend calls `manage_print_order(order_id, action: SHIP, tracking_number, proof_urls)`.

PrintController: 1. Validates that `tracking_number` is present. 2. Updates the order: `{ tracking_number, proof_urls }`. 3. Updates all `Order_Item` records: `print_status = SHIPPED`.

Returns **200 “Order SHIPPED successfully!”**.

21.3.4 Part 4 - Mark Delivered

The Provider clicks “Mark Delivered”. The Frontend calls `manage_print_order(order_id, action: COMPLETE)`.

PrintController updates all `Order_Item` records: `print_status = DELIVERED`.

Returns **200 “Order COMPLETED successfully!”**. The job is closed.

21.4 Key Implementation Details

- The `manage_print_order` endpoint is a single unified action handler that uses the `action` parameter to branch into ACCEPT, SHIP, or COMPLETE logic.
- `print_status` lives on `Order_Item`, not on `Order`. The parent `Order.status` transitions from PENDING to PAID only when actual payment is processed. Print-specific lifecycle states are tracked at the item level.
- `proof_urls` is a `String[]` column on the `Order` table. Providers upload proof photos to S3 and submit their keys as an array.
- Ownership verification (`order.provider_id == req.user.id`) prevents providers from managing orders they are not assigned to.

22 Sequence Diagram - Print Order

UML Type: Sequence Diagram Category: Print Source: docs/diagrams/sequence/print/sequence_print_order

22.1 Overview

This diagram describes how a customer browses available print providers and submits a print job order for a 3D model.

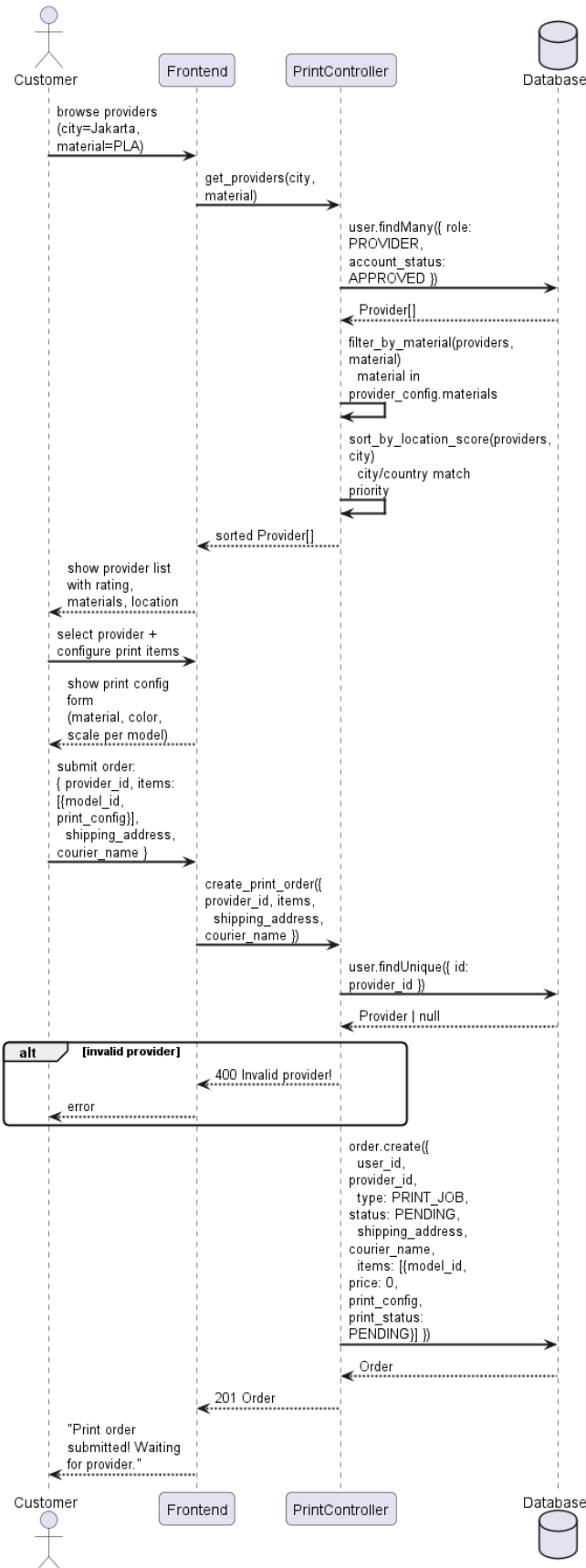


Figure 22: Sequence Diagram - Print Order

22.2 Participants

Participant	Role
Customer	The authenticated user
Frontend	The client
PrintController	Express route handler for print operations
Database	PostgreSQL via Prisma

22.3 Flow

22.3.1 Part 1 - Browse Providers

The Customer wants to get a model physically printed. The Frontend calls `get_providers(city, material)`.

PrintController queries `user.findMany({ role: PROVIDER, account_status: APPROVED })`. It then applies: 1. **Material filter:** Keeps only providers whose `provider_config.materials` array includes the requested material (e.g., “PLA”). 2. **Location sort:** Scores providers based on city/country match and sorts by proximity priority.

PrintController returns sorted providers. The Frontend shows each provider with their rating, supported materials, and location.

22.3.2 Part 2 - Submit Print Order

The Customer selects a provider and configures print items (material, color, scale per model). The Customer submits:

```
{
  provider_id,
  items: [{ model_id, print_config }],
  shipping_address,
  courier_name
}
```

The Frontend calls `create_print_order({ provider_id, items, shipping_address, courier_name })`.

PrintController: 1. Looks up the provider by ID to validate they exist and are approved. 2. If invalid: returns **400 Invalid provider!** 3. Creates the order:

```
order.create({
  user_id, provider_id,
  type: PRINT_JOB, status: PENDING,
  shipping_address, courier_name,
  items: [{ model_id, price: 0, print_config, print_status: PENDING }]
})
```

Note: Print orders start with `price: 0` per item; the financial agreement between customer and provider is handled externally or through a separate billing flow.

PrintController returns **201 Order**. The Frontend confirms: “Print order submitted! Waiting for provider.”

22.4 Error Paths

Condition	HTTP Response
Invalid or unapproved provider	400 Invalid provider

22.5 Key Implementation Details

- Print orders use `Order_Type.PRINT_JOB`. This distinguishes them from digital asset orders (`ASSET`) in all order management views.
- The `provider_config` JSON field on the Provider's User record holds their capabilities (materials, colors, location, pricing). The filtering logic reads this field.
- Unlike digital purchases, print orders do not go through Midtrans immediately. The provider must accept the job first. Payment handling for print jobs may follow a separate flow.
- The `shipping_address` is a JSON field on the Order, containing the delivery details provided by the customer.

23 Sequence Diagram - User Registration

UML Type: Sequence Diagram **Category:** Auth **Source:** docs/diagrams/sequence/auth/sequence_register.puml

23.1 Overview

This diagram describes the registration flow for new users using local credentials (email, username, password). Registration always creates a Customer account. Artist and Provider accounts are upgraded separately through the admin approval flow.

23.2 Participants

Participant	Role
New User	The human actor filling the registration form
Frontend	The Next.js client hosting the form
AuthController	Express route handler for <code>/auth/register</code>
AuthService	Business logic layer
Database	PostgreSQL via Prisma

23.3 Flow

23.3.1 1. Form Submission

The user fills in email, username, and password and submits the form. The Frontend sends these to Auth-Controller.

23.3.2 2. Field Validation

AuthController validates that all three fields are present.

- If any field is missing: returns **400 Missing fields**. The flow terminates.

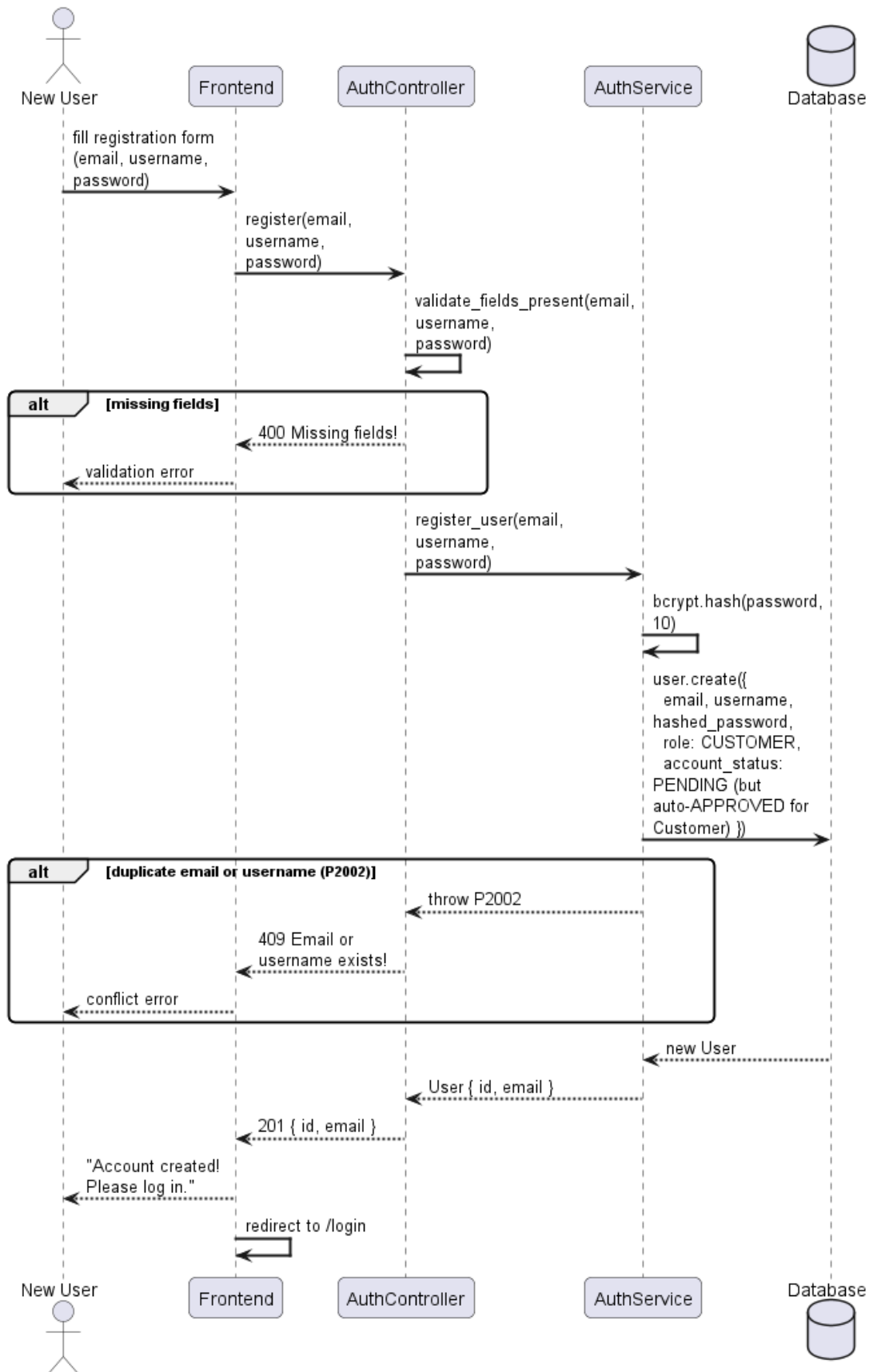


Figure 23: Sequence Diagram - User Registration

23.3.3 3. Password Hashing

AuthService hashes the password using `bcrypt.hash(password, 10)` before any database write.

23.3.4 4. User Creation

AuthService calls `user.create` on the database with: - `email`, `username`, hashed `password` - `role = CUSTOMER` - `account_status = APPROVED` (Customers are auto-approved; they do not require admin review)

If the database raises a unique constraint violation (Prisma error code P2002) due to a duplicate email or username: - AuthController returns **409 Email or username exists**. The flow terminates.

23.3.5 5. Response

AuthController returns **201 Created** with `{ id, email }`. The Frontend displays a success message and redirects to `/login`.

23.4 Error Paths

Condition	HTTP Response
Missing email, username, or password	400 Missing fields
Duplicate email or username	409 Conflict

23.5 Key Implementation Details

- Registration does not issue a session cookie or JWT. The user must log in separately after registering.
- The `role` is hardcoded to `CUSTOMER` in the registration flow. Users who want to become Artists or Providers must apply through the profile settings, which changes their role and sets `account_status = PENDING` for admin review.
- `bcrypt` cost factor is 10, balancing security and performance on the server.

24 Sequence Diagram - Report Submission and Resolution

UML Type: Sequence Diagram **Category:** Admin **Source:** docs/diagrams/sequence/admin/sequence_report.pu

24.1 Overview

This diagram covers the full lifecycle of an abuse report: from a user submitting a report on a piece of content, to an admin reviewing and resolving it. It is split into two phases shown in the same diagram.

24.2 Participants

Participant	Role
User	The authenticated user submitting the report
Frontend	The client
ReportController	Express route handler for report creation

Participant	Role
AdminController	Express route handler for report resolution
Database	PostgreSQL via Prisma

24.3 Flow

24.3.1 Phase 1 - User Submits a Report

Trigger: User clicks “Report” on a model, post, or comment.

1. The Frontend shows a form for the user to select a reason.
2. The user selects a reason and submits.
3. The Frontend calls `create_report({ target_type, reason, [model_id|post_id|comment_id] })`.

ReportController: 1. Validates that `target_type` is one of `MODEL`, `POST`, or `COMMENT`. 2. Validates that `reason` is not empty. 3. Creates a `Report` record with `status = PENDING`.

ReportController returns **201 Created**. The Frontend shows a “Report submitted” confirmation to the user.

24.3.2 Phase 2 - Admin Resolves the Report

Trigger: Admin opens the pending reports list.

1. The Frontend calls `list_reports(status: PENDING)`.
2. AdminController queries the database for all pending reports including reporter details.
3. The Frontend displays the report list to the Admin.

Trigger: Admin clicks “Resolve” on a report.

1. The Frontend calls `resolve_report(report_id, status: REVIEWED)`.
2. AdminController updates the report’s `status` to `REVIEWED`.
3. The Frontend removes the report from the pending queue.

24.4 Error Paths

Condition	HTTP Response
Invalid <code>target_type</code>	400 Bad Request
Empty <code>reason</code>	400 Bad Request

24.5 Key Implementation Details

- The `target_type` enum in the schema is `Report_Target` with values `MODEL`, `POST`, `COMMENT`.
- Depending on the `target_type`, one of `model_id`, `post_id`, or `comment_id` is populated. The others are null.
- The resolution status can be either `REVIEWED` or `DISMISSED` (`Report_Status` enum). The diagram only shows the `REVIEWED` path. The dismiss path follows the same mechanics but sets `status = DISMISSED`.

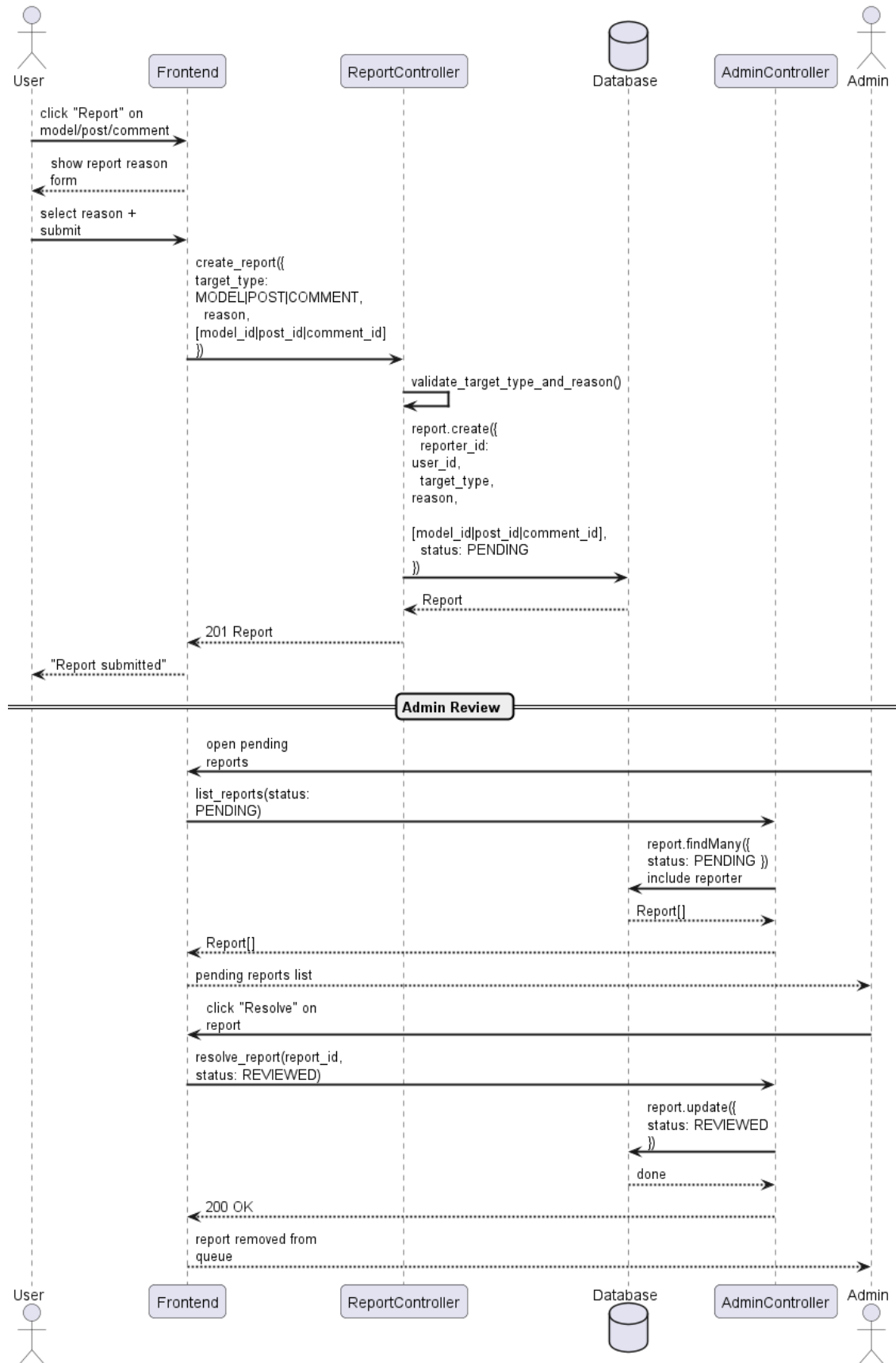


Figure 24: Sequence Diagram - Report Submission and Resolution

- Reports do not automatically trigger any action on the reported content. The admin must take a separate moderation action (delete, ban, etc.) if warranted.
- This diagram shows `resolve_report` handled by the `AdminController`, not the `ReportController`, because only admins can resolve reports. Report creation is available to all authenticated users.

25 Sequence Diagram - Session Hydration

UML Type: Sequence Diagram **Category:** Auth **Source:** docs/diagrams/sequence/auth/sequence_session_hydr

25.1 Overview

Session hydration occurs every time the application mounts or the user refreshes the page. It re-validates the session cookie and populates the frontend's `AuthContext` with fresh user data, ensuring the UI always reflects the current server state.

25.2 Participants

Participant	Role
User	The human actor loading the application
Frontend	The Next.js client running <code>GET /auth/me</code> on mount
AuthController	Express route handler for <code>/auth/me</code>
StorageService	S3 URL presigner
Database	PostgreSQL via Prisma

25.3 Flow

25.3.1 1. App Mount or Page Refresh

Whenever the app loads, the Frontend calls `GET /auth/me`, sending the `3dex_session` cookie automatically via the browser.

25.3.2 2. JWT Validation

`AuthController` runs the auth middleware, which decodes the JWT from the session cookie.

- If no cookie is present or the token is invalid/expired: returns **401 Unauthorized**. The Frontend clears `AuthContext` and shows the guest (logged-out) UI. The flow terminates.

25.3.3 3. Database Lookup

The decoded token contains the user's ID. `AuthController` queries the database to get the full current user record.

- If the user no longer exists (deleted account): returns **404 Not Found**. The Frontend logs out and redirects. The flow terminates.

25.3.4 4. Security Sanitization

`AuthController` strips sensitive fields (`password`, `google_id`) from the user object before building the response.

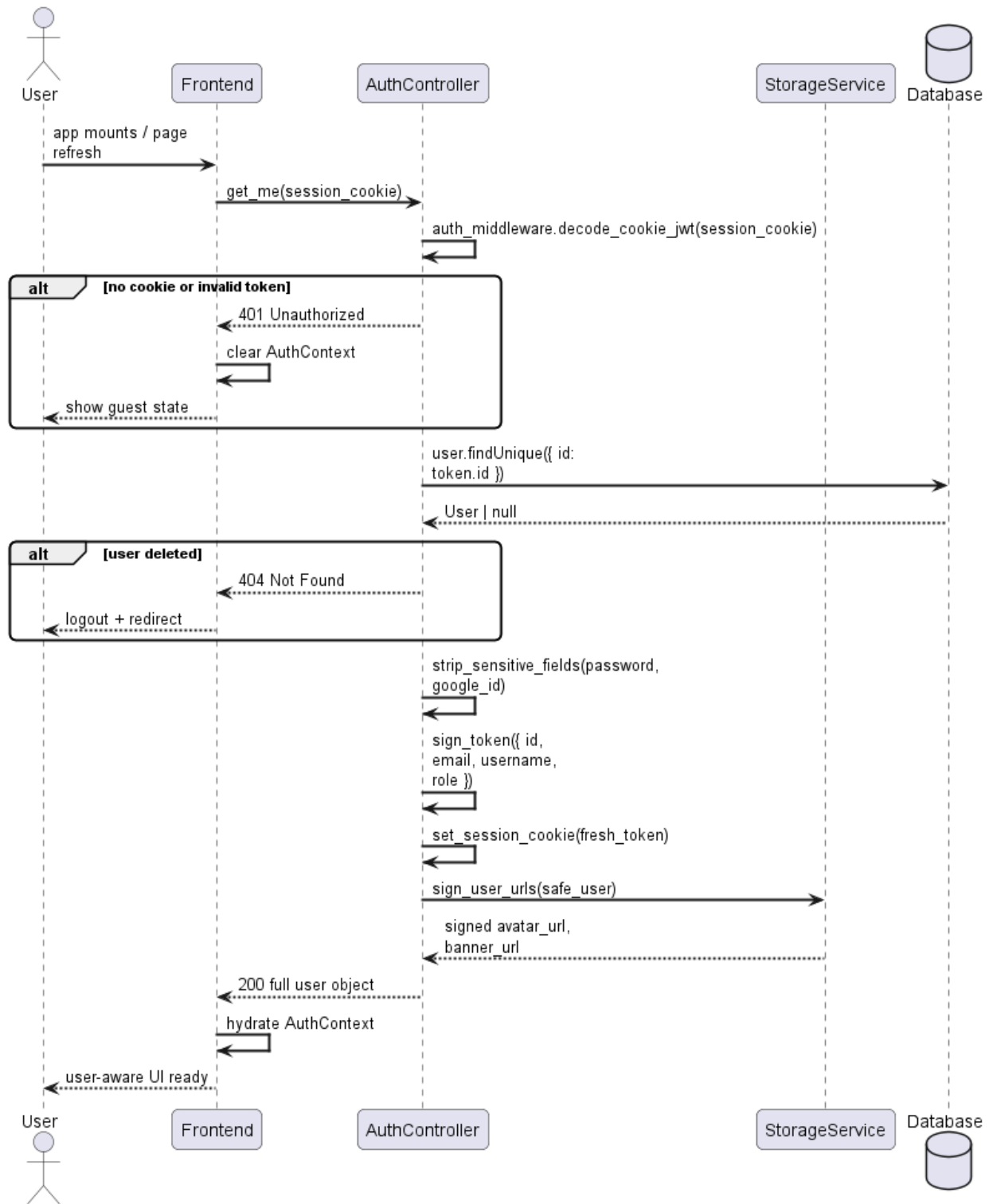


Figure 25: Sequence Diagram - Session Hydration

25.3.5 5. Token Refresh

AuthController signs a fresh JWT and sets a new `3dex_session` cookie. This implements a sliding-window session: every app load extends the session for another 7 days.

25.3.6 6. URL Signing

StorageService generates presigned S3 URLs for the user's `avatar_url` and `banner_url`.

25.3.7 7. Response

AuthController returns **200** with the full sanitized user object, including signed URLs. The Frontend hydrates `AuthContext` and renders the user-aware UI.

25.4 Error Paths

Condition	HTTP Response	Frontend Action
No cookie or invalid JWT	401 Unauthorized	Clear AuthContext, show guest state
User deleted from database	404 Not Found	Logout and redirect

25.5 Key Implementation Details

- This flow is the primary mechanism ensuring the frontend always has fresh user state without requiring an explicit re-login.
- The token refresh on each hydration call means the user effectively stays logged in as long as they visit the app at least once every 7 days.
- `strip_sensitive_fields` removes `password` and `google_id` before the object is returned to the client. These fields are never sent to the browser.
- This is different from a dedicated token refresh endpoint: it simultaneously validates, fetches current data, and re-issues the token in one call.

26 Sequence Diagram - Social Post

UML Type: Sequence Diagram **Category:** Social **Source:** docs/diagrams/sequence/social/sequence_social_po

26.1 Overview

This is the most comprehensive social diagram. It covers the entire community feed experience: loading the feed, creating a new post with image uploads, toggling a like on a post, and adding a comment.

26.2 Participants

Participant	Role
User (Artist / Provider)	The authenticated user
Frontend	The client

Participant	Role
PostController	Express route handler for post operations
StorageController	Express route handler for presigned URL generation
StorageClient (S3)	MinIO backend
Database	PostgreSQL via Prisma

26.3 Flow

26.3.1 Part 1 - Load Feed

The user opens the Community page. The Frontend calls `get_feed_posts()`.

PostController queries `post.findMany(orderBy: created_at desc)` including user details, `_count` for likes and comments, and the current user's likes for each post.

PostController maps additional flags per post: `is_liked` (whether the current user has liked it) and `is_followed` (whether the current user follows the post's author).

Returns the feed to the Frontend.

26.3.2 Part 2 - Create a Post

The user clicks "New Post". The Frontend shows a media upload form.

Step 1: Image Upload

For each selected image, the Frontend requests a presigned PUT URL from StorageController. StorageController returns presigned URLs. The Frontend uploads each image binary directly to MinIO.

Step 2: Post Creation

The Frontend calls `create_post({ caption, media_urls: [keys], is_nsfw })`.

PostController: 1. Checks `user.role`. Only **ARTIST** or **PROVIDER** can create posts. 2. If the role is **CUSTOMER** or **ADMIN**: returns **403 Only Artists and Providers can post!** 3. Validates that `media_urls` is not empty (posts require at least one image). 4. If no media: returns **400 Bad Request**. 5. Calls `post.create({ user_id, caption, media_urls, is_nsfw })`. 6. Returns **201 Post**. The post appears in the feed.

26.3.3 Part 3 - Like / Unlike

The user clicks the heart icon on a post. The Frontend calls `toggle_like(post_id, session_cookie)`.

PostController checks for an existing `Post_Like` record.

If like exists (unlike): 1. Deletes the `Post_Like` record. 2. Decrements `post.like_count`. 3. Returns `{ message: "Unliked" }`.

If no like (like): 1. Creates a `Post_Like` record. 2. Increments `post.like_count`. 3. Returns `{ message: "Liked" }`.

26.3.4 Part 4 - Add a Comment

The user types a comment and submits. The Frontend calls `add_comment(post_id, content)`.

PostController: 1. Validates `content` is not empty. 2. Creates `post.Comment.create({ user_id, post_id, content })`. 3. Increments `post.comment_count`. 4. Returns the `Post_Comment`. The comment appears below the post.

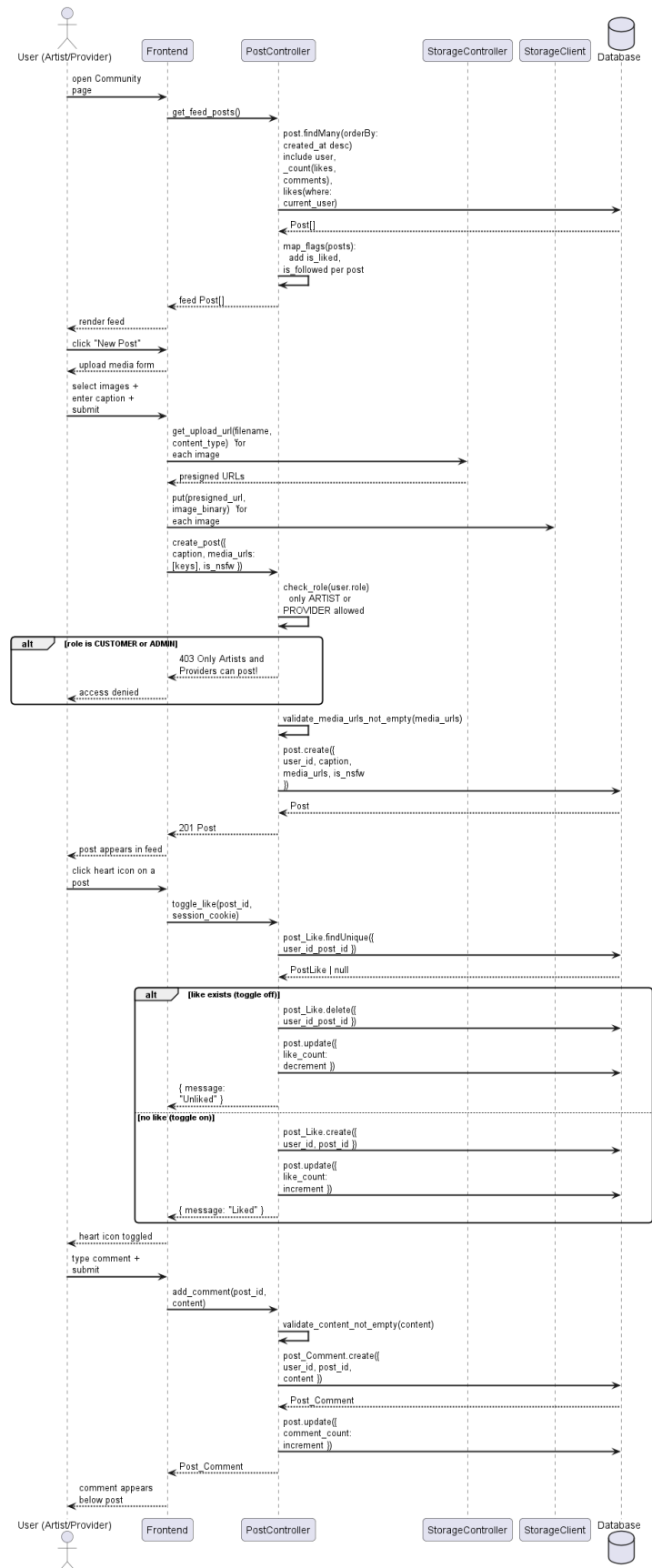


Figure 26: Sequence Diagram - Social Post

26.4 Error Paths

Condition	HTTP Response
Post creation by Customer or Admin	403 Forbidden
Post with no images	400 Bad Request
Empty comment	400 Bad Request

26.5 Key Implementation Details

- `like_count` and `comment_count` are denormalized fields on `Post`. They are updated atomically using Prisma's `increment/decrement` operations alongside the create/delete of the related records.
- The `is_liked` and `is_followed` flags are computed server-side per post, enabling the frontend to render the correct button states without additional requests.
- Admin users cannot create posts. Post creation is restricted to Customers with the Artist or Provider role.

27 Sequence Diagram - User Review

UML Type: Sequence Diagram **Category:** Social **Source:** docs/diagrams/sequence/social/sequence_user_revi

27.1 Overview

This diagram describes how a customer reviews an Artist or Provider as a person (not a model). It covers eligibility checking, submission, and automatic recalculation of the target user's aggregate rating.

27.2 Participants

Participant	Role
Customer	The authenticated user submitting the review
Frontend	The client
ReviewController	Express route handler for user reviews
Database	PostgreSQL via Prisma

27.3 Flow

27.3.1 Part 1 - Eligibility Check

The Customer opens an artist or provider's profile. The Frontend calls `check_review_eligibility(target_user_id, session_cookie)`.

ReviewController:

1. Checks `user_Review.findUnique({ reviewer_id_target_user_id })`.
 - If already reviewed: returns `{ eligible: false, reason: ALREADY_REVIEWED }`. Frontend hides the review button.

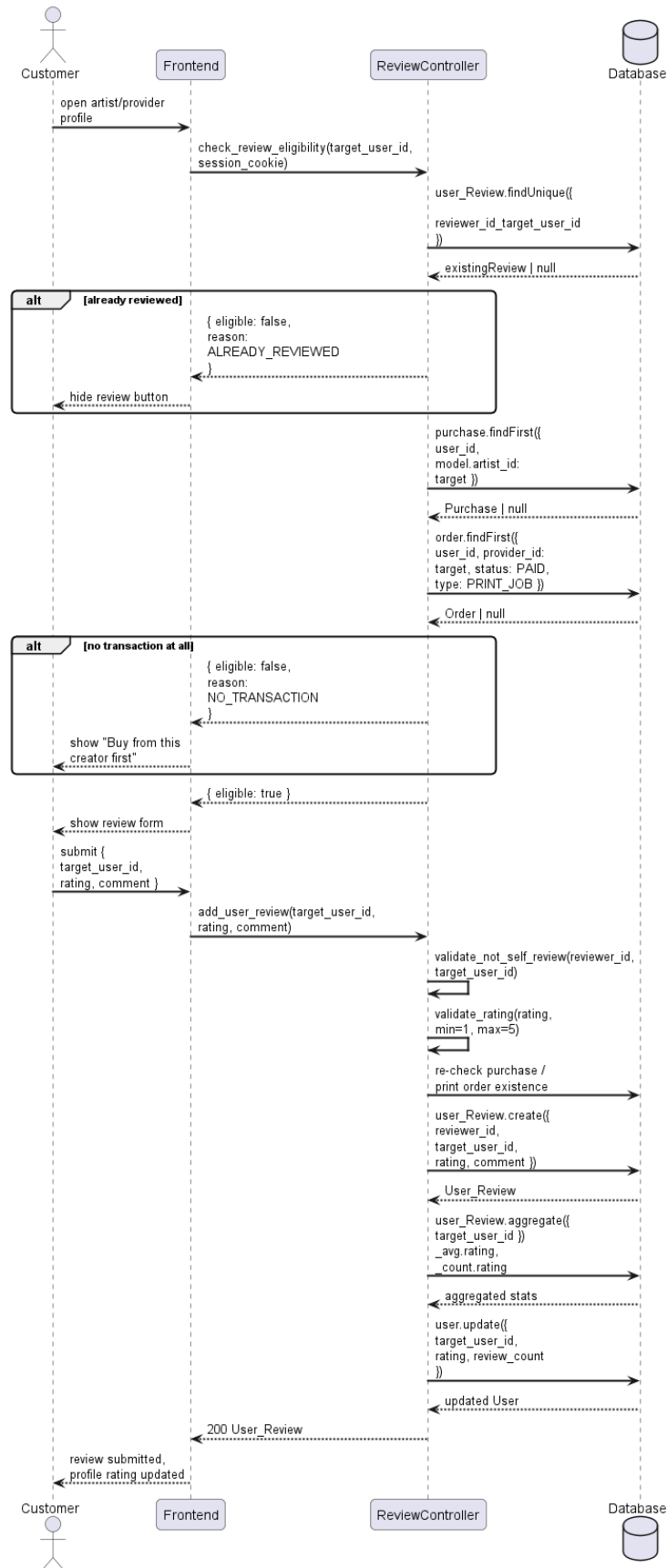


Figure 27: Sequence Diagram - User Review

2. Checks for a prior purchase from the artist: `purchase.findFirst({ user_id, model.artist_id: target_user_id })`
3. Checks for a completed print job from the provider: `order.findFirst({ user_id, provider_id: target_user_id, status: PAID, type: PRINT_JOB })`
 - If neither exists: returns `{ eligible: false, reason: NO_TRANSACTION }`. Frontend shows “Buy from this creator first.”
4. If at least one transaction exists: returns `{ eligible: true }`. Frontend shows the review form.

27.3.2 Part 2 - Review Submission

The Customer submits `{ target_user_id, rating, comment }`. The Frontend calls `add_user_review(target_user_id, rating, comment)`.

ReviewController: 1. Validates that `reviewer_id != target_user_id` (no self-reviews). 2. Validates `rating` is between 1 and 5. 3. Re-checks transaction existence (double-check for security). 4. Calls `user_Review.create({ reviewer_id, target_user_id, rating, comment })`. 5. Calls `user_Review.aggregate({ target_user_id })` to compute `_avg.rating` and `_count.rating`. 6. Calls `user.update({ id: target_user_id, rating: avg, review_count: count })` to update the target’s public rating.

Returns **200 User_Review**. The profile rating is updated immediately.

27.4 Error Paths

Condition	Response
Already reviewed	<code>{ eligible: false, reason: ALREADY_REVIEWED }</code>
No transaction with the target	<code>{ eligible: false, reason: NO_TRANSACTION }</code>
Self-review attempt	Blocked by validation
Duplicate review at DB level	409 Conflict

27.5 Key Implementation Details

- The eligibility check is separate from the submission endpoint. This allows the UI to proactively hide or show the review form without the user attempting a submission.
- **User_Review** is distinct from **Review** (model review). This table tracks peer-to-peer user ratings, not model quality.
- The `rating` and `review_count` fields on **User** are updated after each new review, keeping the aggregate current without requiring expensive full-table aggregations on every profile view.
- The unique constraint on `(reviewer_id, target_user_id)` ensures one review per pair at the database level.

28 Sequence Diagram - Wishlist

UML Type: Sequence Diagram **Category:** Commerce **Source:** docs/diagrams/sequence/commerce/sequence_wis

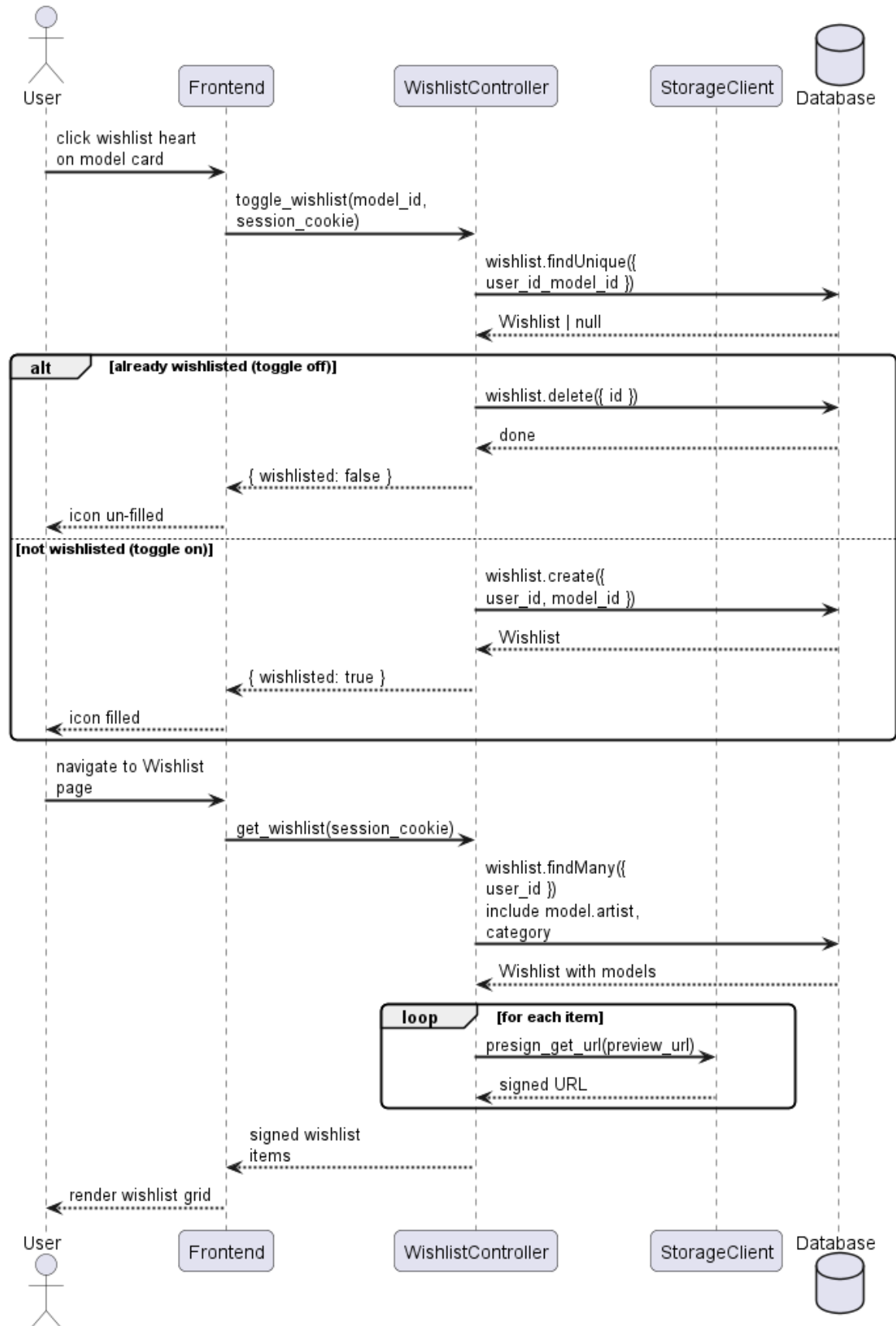


Figure 28: Sequence Diagram - Wishlist

28.1 Overview

This diagram covers two related wishlist interactions: toggling the wishlist state for a model (adding or removing it), and loading the full wishlist page.

28.2 Participants

Participant	Role
User	The authenticated user
Frontend	The client
WishlistController	Express route handler for wishlist operations
StorageClient (S3)	MinIO for presigning preview images
Database	PostgreSQL via Prisma

28.3 Flow

28.3.1 Part 1 - Toggle Wishlist

The user clicks the heart icon on a model card. The Frontend calls `toggle_wishlist(model_id, session_cookie)`.

WishlistController queries `wishlist.findUnique({ user_id: model_id })` to check the current state.

If already wishlisted (remove): 1. `wishlist.delete({ id })` 2. Returns `{ wishlisted: false }`. The Frontend un-fills the heart icon.

If not wishlisted (add): 1. `wishlist.create({ user_id, model_id })` 2. Returns `{ wishlisted: true }`. The Frontend fills the heart icon.

28.3.2 Part 2 - Wishlist Page

The user navigates to the Wishlist page. The Frontend calls `get_wishlist(session_cookie)`.

WishlistController queries `wishlist.findMany({ user_id })` including `model.artist` and `category`.

For each wishlist item, it calls MinIO to presign the model's `preview_url`.

WishlistController returns the signed wishlist items. The Frontend renders a grid of saved models.

28.4 Key Implementation Details

- The toggle is implemented as a lookup-then-create/delete rather than a dedicated toggle endpoint, meaning the state is always read from the database (no client-side optimistic state persistence).
- The `Wishlist` table has a unique constraint on `(user_id, model_id)`, enforced at the database level.
- Wishlist entries are cascade-deleted if the underlying model is deleted.
- Wishlisting is independent of purchasing. A user can wishlist a model they already own.